

# SQLP 과목 III 튜닝 스타일 모의문제 · 12회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

오라클이 데이터 블록을 읽는 두 방식인 한 블록씩 읽는 랜덤 액세스와 여러 블록을 한 번에 읽는 Multiblock I/O가 각각 어떤 대기 이벤트로 집계되고 무엇이 그 단위를 좌우하는지에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인덱스를 경유해 ROWID로 테이블을 한 블록씩 찾아가는 랜덤 액세스는 읽은 블록들이 버퍼 캐시의 여러 슬롯에 흩어져 담기므로 db file scattered read로 대기하고, Full Table Scan처럼 인접한 여러 블록을 한 I/O Call로 요청하는 Multiblock I/O는 한 자리에 이어 담기므로 db file sequential read로 대기한다.
- ② 인덱스 수직 탐색이나 인덱스 경유 테이블 액세스처럼 한 번에 한 블록만 요청하는 Single Block I/O(랜덤 액세스)의 대기는 db file sequential read로 집계되며, 이 방식은 필요한 블록만 골라 읽는 대신 요청 횟수가 읽는 블록 수만큼 늘어난다.
- ③ Full Table Scan이나 Index Fast Full Scan처럼 인접한 여러 블록을 한 번의 I/O Call로 묶어 요청하는 Multiblock I/O의 대기는 db file scattered read로 집계되며, 한 Call로 요청하는 최대 블록 수는 DB\_FILE\_MULTIBLOCK\_READ\_COUNT의 상한을 따른다.
- ④ Multiblock I/O는 한 번의 Call이라 해도 익스텐트 경계를 넘어서까지 읽지는 않으며, 요청 범위 안에 이미 버퍼 캐시에 올라와 있는 블록이 끼어 있으면 그 지점에서 끊어 읽으므로 실제 한 Call의 블록 수가 DB\_FILE\_MULTIBLOCK\_READ\_COUNT보다 작아질 수 있다.

2

데이터를 변경하는 트랜잭션이 수행될 때 버퍼 캐시의 갱신, 변경 전 이미지(Undo), 변경 로그(Redo)가 각각 어떤 역할을 하고 커밋과 어떻게 맞물리는지에 대한 설명으로 가장 적절한 것은?

- ① 블록을 갱신하면 실제 변경은 우선 버퍼 캐시의 블록(Dirty Buffer)에 이뤄지고, 변경 전 이미지는 Undo에, 변경을 재현할 수 있는 로그는 Redo에 기록되며, 갱신된 블록이 데이터파일에 반영되는 시점은 커밋 시점과 별개로 나중에 결정된다.
- ② 블록을 갱신할 때 변경 전 이미지가 Undo에 남는 것은 맞지만, Undo는 이름 그대로 되돌리는 정보이므로 그 이미지는 자기 트랜잭션이 롤백을 선언할 때만 읽히고, 다른 세션이 커밋되지 않은 변경을 배제한 과거 이미지를 재구성해 읽는 읽기 일관성에는 전혀 관여하지 않는다.
- ③ 커밋이 Redo를 디스크에 확정 기록하며 완료되는 것은 맞지만, Redo는 장애 복구를 위한 로그이므로 트랜잭션이 커밋되는 그 순간에 변경분을 한꺼번에 모아 생성되고, 커밋 이전의 중간 변경들이 버퍼 캐시의 블록을 갱신하는 동안에는 로그 버퍼에 Redo가 쌓이지 않는다.
- ④ 변경이 버퍼 캐시의 블록에서 먼저 일어나 Dirty Buffer가 되는 것은 맞지만, 커밋은 그 변경을 영구히 확정하는 선언이므로 Dirty Buffer는 커밋과 동시에 즉시 데이터파일에 기록되어야 하며, 이 디스크 반영이 끝나야 비로소 커밋이 완료된 것으로 처리된다.

법무부 전자비자 심사 시스템의 야간 배치가 접수된 사증신청 400,000건을 순회하며 건마다 심사상태 갱신 UPDATE 1건씩(총 400,000건)을 실행한다. 개발자가 EXECUTE IMMEDIATE로 SQL 문자열을 조립하면서 신청번호·발급국가코드 같은 값을 바인드 변수가 아니라 리터럴 문자열로 이어 붙여 동적 SQL을 만들었기 때문에 실행마다 SQL 텍스트가 조금씩 달라진다. 5개의 심사 세션이 동시에 돌아 응답이 40초로 늘고 대기 이벤트 상위에 library cache: mutex X·cursor: pin S wait on X·shared pool latch가 잡혔다. 아래는 이 배치 세션의 비재귀·재귀 statement를 나눠 집계한 TKPROF이다. 지연의 원인을 직접 지목한 것으로 옳은 것은? (단, 옵티마이저 통계는 최신이고 Shared Pool은 여유가 충분해 에이징으로 인한 강제 재파싱은 없으며, 각 실행은 조건값만 다르고 UPDATE 구조는 동일하다.)

| 아 래                                             |         |          |         |      |          |         |         |
|-------------------------------------------------|---------|----------|---------|------|----------|---------|---------|
| OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS |         |          |         |      |          |         |         |
| Call                                            | Count   | CPU Time | Elapsed | Disk | Query    | Current | Rows    |
| Parse                                           | 400000  | 32.000   | 34.000  | 0    | 0        | 0       | 0       |
| Execute                                         | 400000  | 5.000    | 6.000   | 0    | 800000   | 400000  | 400000  |
| Fetch                                           | 0       | 0.000    | 0.000   | 0    | 0        | 0       | 0       |
| Total                                           | 800000  | 37.000   | 40.000  | 0    | 800000   | 400000  | 400000  |
| Misses in library cache during parse: 380000    |         |          |         |      |          |         |         |
| OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS     |         |          |         |      |          |         |         |
| Call                                            | Count   | CPU Time | Elapsed | Disk | Query    | Current | Rows    |
| Parse                                           | 800000  | 5.000    | 5.000   | 0    | 0        | 0       | 0       |
| Execute                                         | 800000  | 6.000    | 6.000   | 0    | 6400000  | 0       | 0       |
| Fetch                                           | 1600000 | 3.000    | 3.000   | 0    | 9600000  | 0       | 1600000 |
| Total                                           | 3200000 | 14.000   | 14.000  | 0    | 16000000 | 0       | 1600000 |
| Misses in library cache during parse: 18        |         |          |         |      |          |         |         |

- ① 재귀 statement가 Query 논리 읽기 16,000,000과 Fetch 1,600,000회를 쌓아 데이터 디렉터리를 과도하게 읽는 것이 병목이다. Disk 0으로 전부 캐시를 거치는데도 320만 콜이나 반복되는 만큼, DB 버퍼 캐시를 키워 디렉터리 블록을 더 오래 상주시키면 이 논리 읽기가 가벼워져 파싱 지연이 사라진다.
- ② 비재귀 Parse 400,000 : Execute 400,000 = 1:1이고 Parse CPU가 32.0초로 전체 40초를 지배하니, 실행마다 커서를 놓쳐 되풀이되는 소프트파싱 폭주가 병목이다. session\_cached\_cursors를 키우고 커서를 홀드해 Parse 콜 자체를 없애면 library cache: mutex X 대기과 40초가 함께 줄어든다.
- ③ 라이브러리 캐시 미스가 380,000건에 달해 사실상 매 실행이 하드파싱이고, 그 하드파싱이 데이터 디렉터리를 뒤지며 재귀 Call 320만을 유발한 것이 원인이다. 동적 SQL의 리터럴을 바인드 변수로 바꿔 SQL 텍스트를 하나로 고정하면 하드파싱이 소프트파싱으로 바뀌어 40초가 해소된다.
- ④ 비재귀 Execute 400,000콜이 Query 800,000·Current 400,000 논리 읽기로 400,000행을 갱신했고 Execute Elapsed도 6.0초에 이른다. 이 UPDATE 실행 속이 배치 작업의 골격이므로, 인덱스를 보강해 실행계획을 다시 짜면 건당 블록 방문이 줄어 40초가 해소된다.

#### SQL 트레이스로 응답시간을 Service Time과 Wait Time으로 나눠 병목을 진단하는 응답시간 분석(Response Time Analysis)에 대한 설명으로 가장 적절한 것은?

- ① 트레이스에서 Elapsed가 CPU보다 크게 나온다는 것은 세션이 CPU를 놓고 멈춰 있던 시간이 있다는 뜻이고, 블록을 디스크에서 올리는 동안이 바로 그 구간이다. 그 차이는 항상 물리 I/O 대기이므로 대기 이벤트를 따로 나눠 볼 것 없이, 더 빠른 스토리지로 교체하면 응답시간이 그만큼 줄어든다.
- ② 응답시간에서 CPU Time(Service Time)이 큰 비중을 차지한다는 것은 자원을 기다리며 멈춘 Wait Time이 그만큼 작다는 뜻이다. 대기 없이 CPU를 붙들고 일했다면 그것은 곧 SQL이 CPU를 알뜰히 쓰는 효율적인 실행이라는 뜻이므로, 논리 읽기까지 파고들 것 없이 튜닝 대상에서 빼도 된다.
- ③ SQL\*Net message from client(사용자 응답 대기)는 Wait Time에 집계되어 Elapsed를 늘리고 그만큼 Service Time의 비중을 떨어뜨리는 항목이다. 응답시간에 그대로 더해진 이상 서버가 소비한 시간이므로, 서버의 CPU·메모리 자원을 증설해 처리량을 올리면 이 대기도 함께 줄어든다.
- ④ 응답시간은 CPU를 점유해 실제 일한 Service Time과 자원을 기다리며 CPU를 놓고 멈춘 Wait Time의 합이므로, 트레이스에서 CPU와 대기 이벤트별 시간을 먼저 분리해 어느 쪽이 응답시간을 지배하는지 가른 뒤 그 지배 요인에 맞는 처방을 골라야 한다.

해양심층수 취수관리 시스템에서 취수원단위(약 250건)를 드라이빙해 해양심층수취수량(약 4억 건)을 NL 조인으로 2,000,000행 뽑는 SQL의 TKPROF이다. 개발자는 물리 읽기(Disk)가 작으니 캐시 적중이 높아 빠를 것으로 봤는데, 응답이 50초를 넘겨 느리다는 신고가 들어왔다. 아래 트레이스에 대한 해석으로 가장 적절한 것은? (단, 옵티마이저 통계는 최신이고 CPU 자원의 외부 경합은 없으며, 해양심층수취수량의 인덱스 해양심층수취수량\_N1을 경유하는 NL 조인으로만 수행되고 조인 대상 블록은 대부분 이미 버퍼 캐시에 올라와 있어 물리 I/O가 작다.)

| 아 래     |                                                                                   |          |              |        |         |         |         |
|---------|-----------------------------------------------------------------------------------|----------|--------------|--------|---------|---------|---------|
| Call    | Count                                                                             | CPU Time | Elapsed Time | Disk   | Query   | Current | Rows    |
| Parse   | 1                                                                                 | 0.020    | 0.020        | 0      | 0       | 0       | 0       |
| Execute | 1                                                                                 | 0.000    | 0.000        | 0      | 0       | 0       | 0       |
| Fetch   | 2001                                                                              | 40.000   | 50.000       | 200000 | 5000000 | 0       | 2000000 |
| Total   | 2003                                                                              | 40.020   | 50.020       | 200000 | 5000000 | 0       | 2000000 |
| Rows    | Row Source Operation                                                              |          |              |        |         |         |         |
| 2000000 | NESTED LOOPS (cr=5000000 pr=200000 pw=0 time=49000000 us)                         |          |              |        |         |         |         |
| 250     | TABLE ACCESS FULL 취수원단위 (cr=25 pr=0 pw=0 time=700 us)                             |          |              |        |         |         |         |
| 2000000 | TABLE ACCESS BY INDEX ROWID 해양심층수취수량 (cr=4999975 pr=200000 pw=0 time=40000000 us) |          |              |        |         |         |         |
| 2000000 | INDEX RANGE SCAN 해양심층수취수량_N1 (cr=3200000 pr=0 pw=0 time=12000000 us)              |          |              |        |         |         |         |

- ① 물리 읽기 Disk 200,000이 전부 해양심층수취수량 테이블 액세스 노드(pr=200000)에 몰려 있고, 그 노드의 time 40,000,000us가 NESTED LOOPS의 49,000,000us 중 대부분이다. 인덱스 노드는 pr=0이니 50초의 뿌리는 이 테이블 물리 읽기이고, 버퍼 캐시를 키워 200,000블록을 캐시 적중으로 돌리면 응답이 최적화된다.
- ② BCHR이 96.0%로 높아 물리 읽기(Disk 200,000)는 논리 읽기의 4%뿐이고, CPU 40초가 50초의 80%다. 병목은 디스크가 아니라 NL 조인이 해양심층수취수량을 200만 번 방문하며 쌓은 논리 읽기 500만이 CPU를 태우는 것이므로, 조인 방식·액세스 경로를 바꿔 논리 읽기 횟수 자체를 줄여야 한다.
- ③ Fetch의 Elapsed 50.000초에서 CPU 40.000초를 뺀 10초는 세션이 CPU를 놓고 기다린 시간이고, Disk 200,000블록이 그 대기의 실체다. 게다가 Fetch 콜이 2,001회로 잘게 나뉘어 그 대기가 되풀이 누적되므로, 병렬도(DOP)를 높여 디스크 읽기를 여러 서버에 나눠 주면 50초가 개선된다.
- ④ NESTED LOOPS가 2,000,000행을 만드는 데 time 49,000,000us를 쓰는 동안 그 행들이 해시·정렬 작업 영역을 넘겨 temp로 흘렸고, 테이블 노드의 pr=200000이 그렇게 내려간 temp를 되읽은 물리 읽기다. 이 spill이 원인이므로 PGA를 넉넉히 키워 작업 영역을 넓히면 50초가 사라진다.

근로복지공단 산재보상 시스템의 산재요양급여 테이블(약 620만 건)에는 결합 인덱스 요양\_IX = (의료기관코드, 요양개시일자)가 있다. 요양을 담당한 의료기관명을 함께 얻기 위해 산재의료기관 테이블(기본키 의료기관\_PK = 의료기관코드)과 조인하는 다음 SQL의 실행계획이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

## 아 래

```
SELECT g.급여번호, g.요양개시일자, g.승인금액, h.의료기관명
FROM 산재요양급여 g, 산재의료기관 h
WHERE g.의료기관코드 = 'M071'
      AND g.요양개시일자 >= DATE '2026-04-01'
      AND g.요양개시일자 < DATE '2026-07-01'
      AND g.요양구분 = '입원'
      AND h.의료기관코드 = g.의료기관코드;
```

| Id | Pid | Operation                          | ( Cost | Card | Bytes )  |
|----|-----|------------------------------------|--------|------|----------|
| 0  | -   | SELECT STATEMENT                   | ( 418  | 2100 | 138600 ) |
| 1  | 0   | NESTED LOOPS                       | ( 418  | 2100 | 138600 ) |
| 2  | 1   | TABLE ACCESS BY INDEX ROWID 산재요양급여 | ( 410  | 2100 | 94500 )  |
| 3  | 2   | INDEX RANGE SCAN 요양_IX             | ( 26   | 8400 | )        |
| 4  | 1   | TABLE ACCESS BY INDEX ROWID 산재의료기관 | ( 1    | 1    | 21 )     |
| 5  | 4   | INDEX UNIQUE SCAN 의료기관_PK          | ( 0    | 1    | )        |

- ① WHERE의 세 조건(의료기관코드·요양개시일자·요양구분)이 모두 요양\_IX의 인덱스 액세스 조건이 되어 Cost 26의 INDEX RANGE SCAN(Id 3) 단계에서 스캔 범위를 정하고, 그 결과가 Card 8,400이다. Id 2의 Card 2,100은 그 8,400건을 의료기관과 짝지으며 줄어드는 조인 결과 건수다.
- ② 의료기관코드(등치)와 요양개시일자(범위)까지가 요양\_IX의 인덱스 액세스 조건으로 INDEX RANGE SCAN에서 처리(Card 8,400)되고, 인덱스에 없는 요양구분은 TABLE ACCESS BY INDEX ROWID 단계 필터로 걸러져 Card가 2,100으로 줄어든다.
- ③ 요양\_IX의 두 번째 칼럼 요양개시일자가 범위(>=, <) 조건이라 인덱스 정렬이 그 지점에서 끊기므로, 선두 칼럼 의료기관코드의 등치도 인덱스 액세스 조건 자격을 잃고 스캔 후 필터로만 처리된다. Id 3의 Card 8,400은 그렇게 인덱스를 훑은 뒤 두 조건을 필터링해 남은 건수다.
- ④ Id 3 INDEX RANGE SCAN이 반환한 Card 8,400이 곧 이 SQL의 최종 결과 건수이며, Id 2 TABLE ACCESS BY INDEX ROWID 단계는 ROWID로 블록을 읽어 컬럼 값을 채울 뿐이라 건수 감소가 일어나지 않는다. Id 2의 Card 2,100과 Bytes 94,500은 건수가 아니라 블록 접근 비용의 표현이다.

오디오북 플랫폼의 정산 배치에서 오디오북재생계측(약 8,000만 건, 테이블 총 블록 약 400,000)을 작품코드 인덱스로 검색해 특정 작품의 재생 300,000행을 인덱스 순서로 읽어 내려받는 SQL의 TKPROF이다. 인덱스로 잘 타는데도 응답이 50초를 넘긴다. 개발자는 "Fetch가 여러 번이라 인출 왕복이 병목"이라 보고 배열 크기를 더 키우려 하고, 옆자리 동료는 "인덱스가 손익분기점을 넘었으니 풀 스캔으로 바꾸자"고 한다. 아래 트레이스를 배열 크기(Array Processing)·손익분기점·클러스터링 팩터 세 지표로 함께 진단할 때, 실제 병목과 처방을 옳게 종합한 것은? (단, 옵티마이저 통계는 최신이고, 애플리케이션은 이미 배열 인출을 사용하며 인덱스는 작품코드 단일 컬럼으로 구성돼 있다.)

| 아래      |                                                                               |          |              |      |       |         |        |
|---------|-------------------------------------------------------------------------------|----------|--------------|------|-------|---------|--------|
| Call    | Count                                                                         | CPU Time | Elapsed Time | Disk | Query | Current | Rows   |
| Parse   | 1                                                                             | 0.003    | 0.003        | 0    | 0     | 0       | 0      |
| Execute | 1                                                                             | 0.000    | 0.000        | 0    | 0     | 0       | 0      |
| Fetch   | 376                                                                           | 8.000    | 50.000       | 6000 | 61000 | 0       | 300000 |
| Total   | 378                                                                           | 8.003    | 50.003       | 6000 | 61000 | 0       | 300000 |
| Rows    | Row Source Operation                                                          |          |              |      |       |         |        |
| 300000  | TABLE ACCESS BY INDEX ROWID 오디오북재생계측 (cr=61000 pr=6000 pw=0 time=49000000 us) |          |              |      |       |         |        |
| 300000  | INDEX RANGE SCAN 오디오북재생계측_작품_IDX (cr=1000 pr=100 pw=0 time=700000 us)         |          |              |      |       |         |        |

- 배열 크기(ArraySize) 800이라 왕복은 병목이 아니고, 인덱스 경유 61,000블록이 풀 스캔 400,000보다 적어 손익분기점도 미도달이다. 병목은 클러스터링 팩터 밀도 0.20이 만든 테이블 랜덤 액세스이므로, 커버링 인덱스로 테이블 액세스를 없애거나 인덱스 키 순서로 테이블을 재정렬해야 한다.
- 테이블 액세스 노드의 pr 6,000이 그대로 Fetch Disk 6,000으로 잡혔고, 인덱스 노드도 pr 100을 물었으며, 그 테이블 노드의 time 49,000,000us가 전체 50초를 거의 다 삼켰다. CPU는 8.0초뿐이라 나머지는 블록을 디스크에서 올리며 기다린 시간이므로, DB 버퍼 캐시를 키워 이 블록들을 캐시 적중으로 돌리면 랜덤 액세스가 사라진다.
- Fetch 콜이 376회로 나뉘어 300,000행을 실어 나르는 동안 CPU 8.0초에 견줘 Elapsed가 50.0초까지 벌어졌다. 그 42초 격차는 서버가 한 묶음을 넘길 때마다 클라이언트 응답을 기다린 인출 왕복 대기이므로, 배열 크기를 더 키워 Fetch 횟수를 낮추면 왕복이 줄어 50초가 해소된다.
- 테이블 노드의 cr 61,000에서 인덱스 뒀 1,000을 뺀 60,000이 전부 랜덤 블록 방문이고, 읽어 내리는 300,000행도 적은 양이 아니다. 인덱스 경유가 손익분기점을 넘어 랜덤 액세스가 과다해진 것이므로, 인덱스를 버리고 풀 스캔(FULL)으로 바꿔 400,000블록을 순차로 읽으면 50초가 개선된다.

결합 인덱스를 설계할 때 컬럼 순서를 정하는 기준, 컬럼을 추가해 커버링을 노리는 방법, 여러 SQL을 함께 고려하는 절충에 대한 설명으로 가장 적절하지 않은 것은?

- 결합 인덱스의 컬럼 순서는 값의 종류가 많아 선택도가 높은 컬럼을 항상 앞에 두는 것이 원칙이며, 이 한 가지 기준만으로 최적 순서가 결정된다.
- 쿼리가 참조하는 컬럼을 인덱스 뒤쪽에 덧붙여 인덱스만 읽고 처리(커버링)하게 하면 테이블 랜덤 액세스를 없앨 수 있지만, 컬럼 추가는 인덱스 크기와 DML 부하를 늘리므로 필요한 경우로 한정해야 한다.
- 여러 조건 중 =(등치)로 들어오는 컬럼을 범위(BETWEEN·부등호) 조건 컬럼보다 앞에 두어야 인덱스 스캔 시작·종료 지점이 좁혀져, 리프를 훑다가 필터로 버리는 행이 줄어든다.
- 하나의 결합 인덱스로 여러 SQL을 함께 충족시키려면 개별 SQL 각각의 최적이 아니라 전체 SQL 집합의 액세스 패턴과 수행 빈도를 같이 놓고 컬럼 순서·구성을 절충해야 한다.

9

**NL(Nested Loops) 조인이 드라이빙 테이블을 어떻게 반복하고, 내부(inner) 테이블의 인덱스 구성과 부분범위처리와 어떻게 맞물리는지에 대한 설명으로 가장 적절한 것은?**

- ① NL 조인은 드라이빙 건마다 내부 테이블의 인덱스를 반복해서 타므로 건별 탐색 단가가 낮고, 해시 조인처럼 해시 테이블을 미리 만드는 준비 비용도 들지 않는다. 그래서 드라이빙 결과가 수백만 건이어서 대량 데이터를 한꺼번에 조인할 때에도 해시 조인보다 언제나 빠르다.
- ② NL 조인은 드라이빙 테이블에서 조건을 만족한 행을 하나 읽을 때마다 그 값으로 내부 테이블을 반복 탐색하므로, 내부 테이블의 조인 컬럼에 인덱스가 있어 건별로 빠르게 찾을 수 있고 드라이빙 쪽 결과 집합이 크지 않을 때 유리하다.
- ③ NL 조인은 드라이빙 테이블의 조건 만족 행을 하나씩 읽어 내부 테이블을 반복 탐색하는데, 그 반복을 끝까지 돌려 결과 집합 전체를 다 만들어 낸 뒤에야 첫 행을 반환한다. 그래서 먼저 만들어진 일부만 빠르게 돌려주는 부분범위처리에는 해시 조인만 못하다.
- ④ NL 조인에서 반복이 시작되는 지점은 바깥 루프이므로 성능은 드라이빙 테이블을 인덱스로 접근해 조건 만족 행을 빨리 추려 내는지에 달려 있고, 일단 드라이빙 건수만 줄여 두면 내부 테이블은 건마다 Full Scan하더라도 반복 횟수와 무관하게 부담이 크지 않다.

10

**해외송금 정산 배치가 대형 해외송금거래(약 6,800만 건)와 소형 수취은행(약 3,200건)을 수취은행코드로 해시 조인한다. 아래는 그 실행계획으로, 수취은행(Id 2)이 첫 번째 자식으로 올라가 Build Input이 되고 해외송금거래(Id 3)가 두 번째 자식으로 Probe Input이 되었다. 이 실행계획(Build=수취은행, Probe=해외송금거래)을 그대로 낼 수 있는 SQL·힌트로 가장 적절하지 않은 것은?**

아래

| Id  | Operation         | Name   |
|-----|-------------------|--------|
| 0   | SELECT STATEMENT  |        |
| * 1 | HASH JOIN         |        |
| 2   | TABLE ACCESS FULL | 수취은행   |
| 3   | TABLE ACCESS FULL | 해외송금거래 |

- ① 아래 힌트로 수취은행 b를 선행(build)에 두어 그대로 낸다.

```
SELECT /*+ LEADING(b t) USE_HASH(t) */
      b.수취은행명, t.송금액
FROM 수취은행 b, 해외송금거래 t
WHERE b.수취은행코드 = t.수취은행코드;
```

- ② 아래 힌트로 낸다.

```
SELECT /*+ LEADING(t b) USE_HASH(b) */
      b.수취은행명, t.송금액
FROM 수취은행 b, 해외송금거래 t
WHERE b.수취은행코드 = t.수취은행코드;
```

- ③ 아래 힌트로 낸다.

```
SELECT /*+ LEADING(t b) USE_HASH(b) SWAP_JOIN_INPUTS(b) */
      b.수취은행명, t.송금액
FROM 수취은행 b, 해외송금거래 t
WHERE b.수취은행코드 = t.수취은행코드;
```

- ④ 아래 힌트로 낸다.

```
SELECT /*+ LEADING(b t) USE_HASH(t) NO_SWAP_JOIN_INPUTS(t) */
      b.수취은행명, t.송금액
FROM 수취은행 b, 해외송금거래 t
WHERE b.수취은행코드 = t.수취은행코드;
```

**옵티마이저의 서브쿼리 Unnesting(서브쿼리 풀기)과, 그 결과로 만들어진 세미(semi)·안티(anti) 조인에서 어느 집합이 build input이 되는가에 대한 설명으로 가장 적절한 것은?**

- ① 세미 조인으로 Unnesting되면 서브쿼리 쪽 집합은 '짜이 있는지'만 확인당하는 대상이므로 후행(probe) 집합이 되고, 바깥 테이블이 build input으로 적재된다. 해시 조인은 build input을 먼저 올린 뒤 probe 집합을 흘려보내는 구조여서, 확인받는 쪽을 적재하면 확인이 성립하지 않기 때문이다. 그래서 서브쿼리 테이블이 build input으로 쓰이는 형태는 나오지 않는다.
- ② IN 서브쿼리를 그대로 조인으로 바꾸면 서브쿼리 쪽 중복 값 때문에 바깥 행이 여러 번 증식하므로, 옵티마이저는 Unnesting 과정에서 서브쿼리를 먼저 독립적으로 수행해 조인 키의 중복을 없앤 인라인 뷰로 만든 뒤 바깥 집합과 일반 조인한다. 따라서 IN Unnesting의 결과는 세미 조인이 아니라 '일반 조인 + DISTINCT'와 같은 처리로 귀결된다.
- ③ 상관 서브쿼리를 풀지 않고 두면 바깥 행마다 서브쿼리를 되풀이하는 FILTER 방식으로 수행되어 반복 횟수만큼 비용이 붙는다. 서브쿼리 Unnesting은 그 반복 수행을 조인 한 번으로 접는 변환이라 이득이므로, 옵티마이저는 변환 전후 비용을 따로 견주지 않고 문법적으로 변환 가능한 서브쿼리를 모두 세미·안티 조인으로 바꾼다.
- ④ 서브쿼리 Unnesting은 비용 기반 변환이라 옵티마이저가 유리하다고 판단할 때 수행되며, 세미 조인으로 풀린 뒤에는 어느 집합을 build input으로 삼을지도 비용에 따라 정해진다. 그래서 상황에 따라 서브쿼리 쪽이 해시 테이블로 적재돼 바깥 집합을 probe하는 형태(예: HASH JOIN RIGHT SEMI)로도 수행될 수 있다.

**옵티마이저의 쿼리 변환에서 뷰 병합(View Merging), 조건절 Pushing, 조건절 이행(Transitive Predicate Generation)이 뷰를 처리할 때 서로 맞물리는 방식에 대한 설명으로 가장 적절하지 않은 것은?**

- ① 뷰가 병합되면 뷰 경계가 사라져 바깥 조건과 뷰 내부 조건이 한 블록에서 함께 최적화되므로, 조건절 Pushing이라는 별도 단계를 거치지 않고도 바깥 조건이 뷰 안 테이블의 액세스 조건으로 쓰일 수 있다.
- ② 병합되지 못하고 독립 블록으로 남은 뷰라도, 조건절 Pushing으로 바깥 상수 조건을 뷰 안으로 밀어 넣으면 뷰가 스캔·집계할 행을 미리 줄일 수 있으며, GROUP BY 뷰에서 그 조건이 그룹핑 키에 걸리면 집계 전에 걸러져 집계 대상 자체가 줄어든다.
- ③ 조건절 이행으로 뷰 바깥에서 유도된 새 조건은 뷰가 병합될 때에만 뷰 안으로 전달되므로, 병합되지 못한 뷰에는 이행으로 만든 조건이 밀려 들어갈 수 없다.
- ④ 뷰 병합은 뷰 경계를 없애 조인 순서·액세스 경로 탐색의 폭을 넓히는 변환이고 조건절 Pushing은 뷰가 처리할 행을 줄이는 변환이라 목적이 다르며, 옵티마이저는 병합이 막힌 뷰에 대해 차선책으로 Pushing을 적용하기도 한다.

증여세 전자신고 접수 시스템의 조회 모듈이 증여세신고(약 3,400만 건)를 신고유형으로 필터링하는 SELECT를 바인드 변수로 초당 수백 회 발행한다. 신고유형 컬럼에는 도수 분포 히스토그램이 있고, 값이 크게 편중돼 있다 — '일반증여'가 대부분(약 97%)이고 '창업자금'·'가업승계'는 극소수(합 3% 미만)다. 아래 SQL의 커서 공유·바인드 엿보기(Bind Peeking)·적응적 커서 공유(ACS) 동작에 대한 설명으로 가장 적절하지 않은 것은? (단, 오라클이며 CURSOR\_SHARING은 기본값 EXACT이고, 별도로 명시하지 않는 한 세션 파라미터는 기본값이다.)

## 아래

```
-- 신고유형을 바인드 변수로 발행 (값만 :b1 로 바뀐다)
SELECT 신고번호, 증여자, 과세표준, 산출세액
FROM   증여세신고
WHERE  신고유형 = :b1;

-- 발행 예: :b1 = '일반증여'(97%), '창업자금'(극소수), '가업승계'(극소수)
```

- ① 바인드 방식은 :b1 값이 달라도 SQL 텍스트가 하나라 라이브러리 캐시의 부모 커서를 공유하며, 두 번째 발행부터는 만들어 둔 자식 커서를 찾아 재사용하는 소프트파싱으로 처리된다.
- ② 신고유형에 도수 분포 히스토그램이 있으면, 커서를 처음 하드파싱하는 순간 :b1에 실제 담긴 값을 엿보아(Bind Peeking) 그 값의 카디널리티로 실행계획을 세우고 자식 커서에 저장한다.
- ③ 바인드 변수를 쓰면 오라클이 히스토그램을 참조해 실행할 때마다 그 실행의 :b1 값에 맞는 계획으로 정규화해 꽃으므로, 편중이 심한 '가업승계'든 흔한 '일반증여'든 매 실행이 각자의 최적 계획으로 수행된다.
- ④ 적응적 커서 공유(ACS)가 동작하면, 처음 세운 계획이 다른 선택도의 :b1 값에 비효율적임이 드러날 때 커서를 bind-aware로 표시하고 이후 값의 선택도에 따라 별도 자식 커서(다른 계획)를 만들어 나눠 실행할 수 있다.

비용 기반 옵티마이저(CBO)가 여러 컬럼에 각각 조건을 건 쿼리의 카디널리티를 추정할 때, 컬럼별 히스토그램·통계 최신성·컬럼 간 관계가 추정에 관여하는 방식에 대한 설명으로 가장 적절한 것은?

- ① 조건이 걸린 컬럼에 히스토그램이 있으면 값이 편중된 구간의 도수까지 반영되어 그 컬럼의 단일 조건 카디널리티가 정밀해진다. 결합 카디널리티는 이 개별 선택도들을 재료로 삼아 계산되므로, 재료가 정밀해지면 그 컬럼을 포함한 여러 조건의 결합 카디널리티도 함께 정밀해진다.
- ② 옵티마이저 통계가 낡으면 행 수·distinct 값 수·최소최대값이 실제와 떨어져 개별 선택도부터 어긋난다. 통계를 자주 수집해 최신 상태로 유지할수록 이 어긋남이 사라지고, 여러 조건의 선택도를 곱해 추정할 때 생기는 편향까지 줄어들어 결합 카디널리티가 실제에 수렴한다.
- ③ 히스토그램의 버킷 수를 늘려 촘촘히 잡을수록 한 버킷이 담는 값 구간이 좁아져 분포가 세밀하게 기술되고, 그 세밀함이 컬럼 사이의 상관관계까지 더 정밀하게 반영한다. 그래서 버킷 해상도만 충분히 높이면 여러 조건이 함께 걸린 쿼리의 결합 선택도도 정확해진다.
- ④ 여러 컬럼에 각각 조건을 걸면 CBO는 기본적으로 컬럼 간 독립을 가정해 개별 선택도를 곱하므로, 실제로 두 컬럼 값이 강하게 연관돼 있으면(예: 시·군과 우편번호) 결합 카디널리티가 실제와 크게 어긋날 수 있고, 컬럼별 히스토그램만으로는 이 상관을 담지 못한다.

일회용컵 보증금 정산 배치가 당일 반납 집계(반납집계, 약 1,100만 행)를 대상 원장(보증금정산, 약 6,700만 건)에 Upsert하되, 정산상태가 '무효'로 갱신되는 행은 삭제까지 한 문장으로 처리한다. 야간 배치 시간을 줄이려고 아래처럼 /\*+ APPEND PARALLEL(t 8) \*/ 힌트를 붙였다. 보증금정산은 (매장번호, 반납일자)가 기본키이고 LOGGING 속성이다. 이 MERGE의 대량 DML·병렬·직접경로 동작에 대한 설명으로 가장 적절한 것은? (단, 오라클이며 세션에서 병렬 DML을 별도로 활성화(ENABLE PARALLEL DML)하지 않았고, 데이터베이스는 FORCE LOGGING이 아니다.)

## 아래

```

MERGE /*+ APPEND PARALLEL(t 8) */ INTO 보증금정산 t
USING 반납집계 s
ON (t.매장번호 = s.매장번호 AND t.반납일자 = s.반납일자)
WHEN MATCHED THEN
    UPDATE SET t.정산금액 = s.집계금액, t.정산상태 = s.집계상태
    DELETE WHERE (s.집계상태 = '무효')
WHEN NOT MATCHED THEN
    INSERT (매장번호, 반납일자, 정산금액, 정산상태)
VALUES (s.매장번호, s.반납일자, s.집계금액, s.집계상태);

```

- ① 문장에 PARALLEL(t 8) 힌트가 붙어 있으므로 스캔뿐 아니라 대상 테이블을 바꾸는 UPDATE·INSERT·DELETE 부분까지 8 병렬로 수행되어, 세션에서 병렬 DML을 따로 켜지 않아도 갱신·삭제가 병렬화된다.
- ② APPEND로 인한 직접경로 방식은 MERGE의 INSERT(미매칭) 부분에만 적용되어 HWM 위 새 블록에 순차로 적재되며, 매칭 행에 대한 UPDATE·DELETE 부분은 직접경로가 아니라 일반(버퍼 캐시 경유) 방식으로 처리된다.
- ③ 직접경로 적재는 버퍼 캐시를 우회하므로, 대상 테이블이 LOGGING 속성이더라도 이 INSERT된 데이터에 대한 리두는 생성되지 않아 리두 부하가 사라진다.
- ④ 직접경로로 적재하는 동안에도 다른 세션은 같은 테이블에 일반 INSERT를 병행할 수 있고, 적재 중인 트랜잭션도 커밋 전에 그 테이블을 조회해 방금 넣은 행을 곧바로 확인할 수 있다.

간편이체 정산 시스템의 두 대용량 테이블 이체거래(약 9,300만 건)와 제휴처정산(약 4,100만 건)을 제휴처번호로 조인해 제휴처별 정산액을 집계한다. 두 테이블은 아래 DDL처럼 조인 키인 제휴처번호로 같은 해시 파티션 8개로 나뉘어 있다. SELECT ... FROM 이체거래 a, 제휴처정산 b WHERE a.제휴처번호 = b.제휴처번호 ... 형태 조인의 Partition-Wise Join 성립 여부와 데이터 재분배에 대한 설명으로 가장 적절한 것은? (단, 오라클이며 두 테이블의 제휴처번호는 같은 NUMBER 타입이다.)

## 아래

```
CREATE TABLE 이체거래 (
  거래번호    NUMBER,
  제휴처번호  NUMBER,
  이체일자    DATE,
  이체금액    NUMBER
)
PARTITION BY HASH (제휴처번호) PARTITIONS 8;

CREATE TABLE 제휴처정산 (
  정산번호    NUMBER,
  제휴처번호  NUMBER,
  정산일자    DATE,
  정산금액    NUMBER
)
PARTITION BY HASH (제휴처번호) PARTITIONS 8;
```

- ① 제휴처번호가 아니라 이체일자로 조인해도, 두 테이블이 해시 파티셔닝돼 있기만 하면 대응 파티션끼리 짝지어 조인하는 Partition-Wise Join이 성립해 데이터 재분배가 없다.
- ② 한쪽 테이블만 제휴처번호로 파티셔닝돼 있어도 옵티마이저가 나머지를 그대로 둔 채 파티션 단위로 조인하므로, 이 경우에도 조인을 위한 데이터 재분배(PX SEND)는 일어나지 않는다.
- ③ 두 테이블이 조인 키 제휴처번호로 같은 해시 함수·같은 파티션 수(8개)로 나뉘어 있으므로, 대응하는 파티션쌍끼리 독립적으로 조인하는 전체(Full) Partition-Wise Join이 성립해 조인을 위한 데이터 재분배를 최소화한다.
- ④ 조인 키에 NVL(제휴처번호, 0) 같은 가공을 씌워 양쪽을 맞춰도 값 자체는 보존되므로, 파티션쌍이 그대로 유지되어 전체 Partition-Wise Join이 성립한다.

조달청 납품 정산 야간 배치가 납품이행실적(약 2억 6천만 건)을 업체코드로 공급업체(약 5,200건)와, 품목코드로 조달품목(약 90만 건)과 각각 조인해 업체·품목분류별 납품 건수를 집계한다. 세 테이블 어느 쪽도 조인 키 기준 파티션이 아니다. 아래 병렬 실행계획에서 두 조인이 병렬 서버(Slave) 사이에 각각 어떤 분배 방식으로 수행되는지 직접 지목한 설명으로 가장 적절한 것은?

## 아래

| Id   | Operation           | Name     | TQ    | IN-OUT |
|------|---------------------|----------|-------|--------|
| 0    | SELECT STATEMENT    |          |       |        |
| 1    | PX COORDINATOR      |          |       |        |
| 2    | PX SEND QC (RANDOM) | :TQ10003 | Q1,03 | P->S   |
| 3    | HASH GROUP BY       |          | Q1,03 | PCWP   |
| * 4  | HASH JOIN           |          | Q1,03 | PCWP   |
| 5    | PX RECEIVE          |          | Q1,03 | PCWP   |
| 6    | PX SEND HASH        | :TQ10002 | Q1,02 | P->P   |
| * 7  | HASH JOIN           |          | Q1,02 | PCWP   |
| 8    | PX RECEIVE          |          | Q1,02 | PCWP   |
| 9    | PX SEND BROADCAST   | :TQ10000 | Q1,00 | P->P   |
| 10   | PX BLOCK ITERATOR   |          | Q1,00 | PCWC   |
| * 11 | TABLE ACCESS FULL   | 공급업체     | Q1,00 | PCWP   |
| 12   | PX BLOCK ITERATOR   |          | Q1,02 | PCWC   |
| * 13 | TABLE ACCESS FULL   | 납품이행실적   | Q1,02 | PCWP   |
| 14   | PX RECEIVE          |          | Q1,03 | PCWP   |
| 15   | PX SEND HASH        | :TQ10001 | Q1,01 | P->P   |
| 16   | PX BLOCK ITERATOR   |          | Q1,01 | PCWC   |
| * 17 | TABLE ACCESS FULL   | 조달품목     | Q1,01 | PCWP   |

- ① 업체코드 조인(Id 7)은 두 입력이 :TQ10000과 Q1,02 흐름을 타고 올라와 조인 키 해시로 공급업체(Id 11)와 납품이행실적(Id 13)을 양쪽 재분배하는 hash-hash이고, 품목코드 조인(Id 4)은 약 90만 건뿐인 소형 조달품목(Id 17)을 Id 15에서 전 서버로 복제해 붙이는 broadcast 분배다.
- ② 업체코드 조인(Id 7)에서 :TQ10000을 타는 PX SEND BROADCAST(Id 9)가 대형 납품이행실적(Id 13)을 전 서버에 복제하고, 약 5,200건뿐인 소형 공급업체(Id 11)는 PX BLOCK ITERATOR(Id 10)로 자기 블록 범위만 읽어 로컬에서 조인하는 broadcast 분배다. 큰 쪽을 뿌려야 각 서버가 작은 쪽 전량과 맞붙는다.
- ③ 업체코드 조인(Id 7)은 소형 공급업체(Id 11)를 PX SEND BROADCAST(Id 9)로 복제하고 납품이행실적(Id 13)은 재분배 없이 로컬로 읽는 broadcast이며, 품목코드 조인(Id 4)은 그 결과(Id 6)와 조달품목(Id 15)을 양쪽 PX SEND HASH로 재분배하는 hash-hash 분배다.
- ④ 업체코드 조인(Id 7)이 소형 공급업체(Id 11)를 PX SEND BROADCAST(Id 9)로 복제하는 broadcast인 것처럼, 품목코드 조인(Id 4)에서도 약 90만 건인 조달품목(Id 17)이 Id 15에서 PX SEND BROADCAST로 전 서버에 복제된 뒤 Id 6으로 올라온 결과와 로컬로 조인된다. 두 조인이 다 broadcast 분배다.

분석함수(윈도우 함수)의 정렬·계산 시점과 집합 연산자(UNION/UNION ALL)의 정렬·중복 처리에 대한 설명으로 가장 적절하지 않은 것은?

- ① 순위 함수의 OVER(ORDER BY ...)는 순위를 매길 때 쓰는 정렬 기준이므로, 그 정렬 덕분에 바깥에 별도의 ORDER BY를 붙이지 않아도 최종 결과가 그 순위 기준으로 정렬되어 출력된다.
- ② 집계형 분석함수는 OVER()에 ORDER BY가 없으면 파티션 전체가 하나의 프레임이 되어 파티션 총합이 모든 행에 같은 값으로 붙지만, ORDER BY를 더하면 기본 프레임이 현재 행까지로 좁혀져 누적합(running total)이 된다.
- ③ 분석함수는 WHERE·GROUP BY·HAVING이 처리된 뒤 거의 마지막 단계에서 계산되므로, 같은 쿼리 블록의 WHERE 절에서는 분석함수 결과를 곧바로 조건으로 걸 수 없고 인라인 뷰로 감싼 뒤 바깥에서 걸러야 한다.
- ④ 집합 연산자 중 UNION은 결과의 중복을 제거하려고 Sort Unique(또는 Hash Unique)를 거치지만 UNION ALL은 그 단계 없이 두 입력을 이어 붙이며, 어느 쪽이든 두 브랜치의 SELECT 컬럼 개수와 대응 타입은 호환되어야 결합된다.

19

지역사랑상품권 판매 관리의 상품권판매(약 2,900만 건)에 취급점 A·B·C의 당일판매액이 각각 400·500·600(합 1,500)으로 들어 있다. 조회 세션 TX2가 격리수준을 SERIALIZABLE로 지정하고 트랜잭션을 시작해 첫 조회로 스냅샷을 확정한 뒤(t1), 갱신 세션 TX1이 UPDATE 두 건을 발행해 하나만 커밋한다. 그 후 TX2가 같은 트랜잭션 안에서 SELECT SUM(당일판매액)을 다시 발행한다(t4). TX2가 t4에서 반환하는 SUM과, 만약 TX2가 READ COMMITTED였다면 t4에서 반환했을 SUM의 조합으로 적절한 것은? (단, 오라클이며 관련 Undo는 정상적으로 남아 있다.)

## 아래

| 시각 | TX1 (갱신 세션)                                                         | TX2 (조회 세션)                                                                      |
|----|---------------------------------------------------------------------|----------------------------------------------------------------------------------|
| t1 |                                                                     | SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;<br>SELECT ...; -- 첫 조회로 스냅샷 SCN 확정 |
| t2 | UPDATE 상품권판매 SET 당일판매액=450<br>WHERE 취급점='A'; (A 400→450)<br>COMMIT; | -- 커밋 완료                                                                         |
| t3 | UPDATE 상품권판매 SET 당일판매액=560<br>WHERE 취급점='B'; (B 500→560)            | -- 미커밋 (커밋 안 함)                                                                  |
| t4 |                                                                     | SELECT SUM(당일판매액)<br>FROM 상품권판매; (이 값을 구함)                                       |

초기: A=400, B=500, C=600 (합 1,500)

- ① SERIALIZABLE 1,550 / READ COMMITTED 1,500
- ② SERIALIZABLE 1,500 / READ COMMITTED 1,610
- ③ SERIALIZABLE 1,610 / READ COMMITTED 1,550
- ④ SERIALIZABLE 1,500 / READ COMMITTED 1,550

20

공영자전거 대여 시스템(SQL Server)의 대여현황(약 5,800만 건)은 기본 READ COMMITTED(잠금 기반, RCSI = OFF)로 운영한다. 갱신 세션 60이 자전거 B1004 행을 UPDATE해 X 잠금을 쥔 채 커밋하지 않았고, 조회 세션 72가 같은 B1004 행을 SELECT로 읽으려 한다. 이때 sys.dm\_tran\_locks 등을 조회한 결과가 아래와 같다. 이 상황에 대한 해석으로 가장 적절한 것은?

## 아래

| [ sys.dm_tran_locks / 요청 상태 - 대여현황 (RCSI = OFF) ] |              |               |                |             |
|---------------------------------------------------|--------------|---------------|----------------|-------------|
| session_id                                        | request_mode | resource(KEY) | request_status | 대기 대상       |
| 60                                                | X            | 자전거:B1004     | GRANT          | -           |
| 72                                                | S            | 자전거:B1004     | WAIT           | → 60 보유분 대기 |

\* KEY = 행(키) 잠금, X = 배타 잠금, S = 공유 잠금  
 \* 60은 B1004 갱신 후 미커밋, 72는 같은 행을 SELECT 시도  
 \* 60은 72(또는 다른 세션)가 쥔 어떤 잠금도 기다리지 않음

- ① 60과 72가 서로를 기다리는 순환 대기라 교착 상태이며, SQL Server의 락 모니터가 롤백 비용이 작은 쪽을 victim(요류 1205)으로 선정·롤백한다.
- ② SNAPSHOT/RCSI 계열의 행 버전 관리에서는 갱신 세션도 잠금을 잡지 않으므로, 60의 UPDATE도 X 잠금 없이 수행되어 애초에 이런 대기가 생기지 않는다.
- ③ 60이 B1004에 X 잠금을 보유한다는 사실만으로 조회 차단이 결정되므로, RCSI를 켜든 끄든 72의 대기는 똑같이 발생한다.
- ④ RCSI가 꺼져 있어 72가 공유(S) 잠금을 요청하다 60의 X 잠금에 막혀 대기하는 단순 블로킹이며, RCSI를 켜면 72는 잠금 대신 버전 저장소의 마지막 커밋 버전을 읽어 블로킹 없이 진행한다.

## 정답 및 해설

### ■ 정답 모아보기

|       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. ①  | 2. ①  | 3. ③  | 4. ④  | 5. ②  |
| 6. ②  | 7. ①  | 8. ①  | 9. ②  | 10. ② |
| 11. ④ | 12. ③ | 13. ③ | 14. ④ | 15. ② |
| 16. ③ | 17. ③ | 18. ① | 19. ④ | 20. ④ |

### 문제 1. 정답 ①

두 대기 이벤트의 이름이 직관과 어긋나 반대로 볼기 쉽습니다. 어떤 방식이 어느 이벤트로 집계되는지를 갈라 봐야 합니다.

Single Block I/O (랜덤 액세스) : 한 번에 한 블록씩 요청

- 인덱스 순서를 따라 순차적으로 한 블록씩 방문
- 대기 이벤트 = db file sequential read

Multiblock I/O (Full Scan류) : 인접한 여러 블록을 한 Call로 요청

- 여러 블록이 버퍼 캐시의 흩어진 슬롯에 담김
- 대기 이벤트 = db file scattered read

이벤트 이름은 읽는 방식이 아니라 버퍼에 담기는 모습에서 왔습니다. 한 블록씩 인덱스를 따라 순차적으로 읽는 랜덤 액세스가 **db file sequential read**, 여러 블록을 한 번에 읽어 캐시 슬롯에 흩뿌리는 Multiblock I/O가 **db file scattered read**입니다.

①은 이 둘을 정반대로 뒤집었습니다. 한 블록씩 찾아가는 랜덤 액세스를 **db file scattered read**로, 여러 블록을 묶어 읽는 Multiblock I/O를 **db file sequential read**로 서술했지만, 실제 집계는 그 반대입니다. 이벤트 이름의 유래(순차 방문 vs 흩어진 적재)를 읽기 단위와 어긋나게 맞바꾼 서술입니다.

②·③·④는 옳습니다. Single Block I/O가 **db file sequential read**로 집계된다는 점(②), Multiblock I/O가 **db file scattered read**이며 블록 수가 **DB\_FILE\_MULTIBLOCK\_READ\_COUNT**의 상한을 따른다는 점(③), 익스텐트 경계와 캐시 적중 블록 때문에 실제 Call 블록 수가 상한보다 작아질 수 있다는 점(④)이 모두 실제 I/O 동작에 부합합니다.

| 선택지 | 판정 | 이유                                                                                                                                                           |
|-----|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | 한 블록씩 찾아가는 랜덤 액세스는 <b>db file sequential read</b> , 여러 블록을 한 Call로 읽는 Multiblock I/O는 <b>db file scattered read</b> 로 집계됩니다. 두 이벤트 이름을 읽기 단위와 정반대로 맞바꾼 서술입니다 |
| ②   | ○  | 인덱스 수직 탐색·인덱스 경유 테이블 액세스는 한 번에 한 블록만 요청하는 Single Block I/O로 <b>db file sequential read</b> 로 대기하며, 요청 횟수가 읽는 블록 수만큼 늘어납니다                                    |
| ③   | ○  | Full Table Scan·Index Fast Full Scan의 Multiblock I/O는 <b>db file scattered read</b> 로 집계되고, 한 Call 최대 블록 수는 <b>DB_FILE_MULTIBLOCK_READ_COUNT</b> 의 상한을 따릅니다  |
| ④   | ○  | Multiblock I/O는 익스텐트 경계를 넘어 읽지 않고, 요청 범위에 캐시 적중 블록이 끼면 그 지점에서 끊으므로 실제 Call 블록 수가 상한보다 작아질 수 있습니다                                                             |

#### ■ 이 문제의 핵심

- 랜덤 액세스(Single Block I/O)는 한 블록씩 인덱스를 따라 순차적으로 읽어 **db file sequential read**로 집계됩니다.
- Multiblock I/O는 인접 여러 블록을 한 Call로 읽어 캐시 슬롯에 흩어 담으므로 **db file scattered read**로 집계됩니다.
- 두 이벤트 이름은 읽기 방식이 아니라 버퍼에 담기는 모습에서 유래하므로 직관과 어긋나 맞바꾸기 쉽습니다.
- Multiblock I/O의 한 Call 블록 수는 **DB\_FILE\_MULTIBLOCK\_READ\_COUNT**의 상한을 따르되, 익스텐트 경계와 캐시 적중 블록 때문에 실제로는 그보다 작아질 수 있습니다.

■ 한 줄 정리 한 블록씩 읽는 랜덤 액세스가 **db file sequential read**, 여러 블록을 한 번에 읽는 Multiblock I/O가 **db file scattered read**인데, 이 둘을 서로 바꾼 ①은 대기 이벤트를 읽기 단위와 정반대로 붙인 서술입니다.

### 문제 2. 정답 ①

세 요소가 커밋과 맺는 관계를 갈라 봐야 합니다. 이름에서 오는 반사적 연상이 함정입니다.

변경 발생 시

버퍼 캐시 : 실제 블록 갱신 → Dirty Buffer (디스크 반영은 지연)

Undo : 변경 '전' 이미지 저장 → 롤백 + 다른 세션의 읽기 일관성 재구성

Redo : 변경을 재현할 로그 → 변경이 날 때마다 로그 버퍼에 계속 축적

커밋 시

Redo(로그)만 디스크에 확정 기록 → 커밋 완료 (WAL / Fast Commit)

Dirty Buffer의 데이터파일 반영은 커밋과 무관하게 나중에

트랜잭션이 데이터를 바꾸면 실제 갱신은 버퍼 캐시의 블록에서 일어나 Dirty Buffer가 되고, 그 블록이 데이터파일로 내려가는 시점은 커밋과 별개로 나중에 정해집니다. 커밋이 보장하는 것은 데이터 블록의 디스크 기록이 아니라 Redo(로그)의 확정 기록입니다(선기록 로그·Fast Commit). Undo에는 변경 전 이미지가, Redo에는 변경을 재현할 로그가 각각 남습니다. ①은 이 세 관계를 모두 옳게 서술했습니다. 이 구조는 오라클(Undo·Redo)과 SQL Server(버전 저장소·트랜잭션 로그)에 공통으로 적용됩니다.

②·③·④는 앞머리에 참인 전제(Undo에 변경 전 이미지가 남는다·커밋이 Redo를 확정 기록한다·변경이 버퍼 캐시에서 먼저 일어난다)를 깔아 두고, 뒷문장에서 이름에서 온 반사적 연상을 결론으로 굳혀 틀렸습니다. Undo를 "되돌리기니까 롤백 전용"으로(②), Redo를 "복구 로그니까 커밋 때만 생성"으로(③), 커밋을 "확정이니까 데이터 디스크 기록"으로(④) 연상했지만, 실제 동작은 각각 다릅니다.

| 선택지 | 판정 | 이유                                                                                                                                                             |
|-----|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ○  | 변경은 버퍼 캐시 블록(Dirty Buffer)에 먼저 이뤄지고 데이터파일 반영은 커밋과 별개로 지연되며, Undo에 변경 전 이미지·Redo에 변경 로그가 남는다는 세 관계를 모두 옳게 짚었습니다                                                 |
| ②   | ×  | 변경 전 이미지가 Undo에 남는다는 앞부분은 맞지만, 그 이미지는 롤백뿐 아니라 다른 세션이 커밋 전 변경을 배제하고 과거 이미지를 재구성해 읽는 읽기 일관성에도 쓰입니다. 참인 전제에서 "롤백 전용"이라는 결론으로 건너뛴 반사적 연상입니다                        |
| ③   | ×  | 커밋이 Redo를 디스크에 확정 기록한다는 앞부분은 맞지만, Redo는 커밋 순간에 한꺼번에 생기는 것이 아니라 커밋 이전 중간 변경이 날 때마다 로그 버퍼에 계속 축적됩니다. "복구 로그=커밋 때만 생성"은 확정 기록 시점과 생성 시점을 뭉개 반사적 연상입니다             |
| ④   | ×  | 변경이 버퍼 캐시 블록에서 먼저 일어나 Dirty Buffer가 된다는 앞부분은 맞지만, 커밋은 Redo(로그)의 확정 기록으로 완료되며 Dirty Buffer의 데이터파일 반영을 요구하지 않습니다(선기록 로그·Fast Commit). "커밋=데이터 디스크 기록"은 반사적 연상입니다 |

#### ▪ 이 문제의 핵심

1. 변경은 버퍼 캐시의 블록(Dirty Buffer)에서 먼저 이뤄지고, 데이터파일 반영은 커밋과 별개로 지연됩니다.
2. Undo는 변경 전 이미지로, 롤백뿐 아니라 다른 세션의 읽기 일관성 재구성에도 쓰입니다.
3. Redo는 변경이 날 때마다 로그 버퍼에 계속 축적되고, 커밋은 그 로그를 디스크에 확정하는 시점입니다.
4. 커밋 완료의 조건은 데이터 블록 기록이 아니라 Redo의 확정 기록입니다(선기록 로그·Fast Commit). 이 구조는 두 DBMS에 공통입니다.

▪ 한 줄 정리 변경은 버퍼 캐시에서 먼저 나고 데이터파일 반영은 지연되며 커밋은 Redo(로그) 기록으로 완료되는데, Undo를 롤백 전용으로·Redo를 커밋 때만 생성으로·커밋을 데이터 디스크 기록으로 연상한 나머지 셋은 이름에서 온 반사적 연상이고, 세 관계를 옳게 짚은 ①이 정답입니다.

### 문제 3. 정답 ③

동적 SQL이 실행마다 리터럴 값을 이어 붙이므로 SQL 텍스트가 매번 달라지고, 커서는 라이브러리 캐시에서 공유되지 못합니다. 그러면 파싱은 대부분 하드파싱이 됩니다. 먼저 하드파싱율을 쟁니다.

$$\begin{aligned}\text{하드파싱율} &= \text{라이브러리 캐시 미스(비재귀)} \div \text{비재귀 Parse 콜} \\ &= 380,000 \div 400,000 = 95.0\%\end{aligned}$$

→ 거의 매 실행이 계획을 새로 생성하는 하드파싱

하드파싱은 계획을 새로 만들 때마다 데이터 딕셔너리(권한·통계·객체 정보)를 조회하는 재귀 statement를 대량으로 유발합니다. 재귀 축을 봅시다.

$$\begin{aligned}\text{재귀 Call 비율} &= \text{재귀 Total 콜} \div \text{비재귀 Total 콜} = 3,200,000 \div 800,000 = 4배 \\ \text{재귀 비중} &= 3,200,000 \div (3,200,000 + 800,000) = 80.0\% \\ \rightarrow &\text{하드파싱이 딕셔너리 재귀 SQL 320만 콜을 낳았다(하드파싱의 부산물)}\end{aligned}$$

시간의 정체도 파싱 축입니다. 비재귀 Parse에 CPU 32.0초가 잡혀 있습니다.

$$\begin{aligned}\text{Parse CPU 비중(비재귀)} &= 32.0 \div 40.0 \times 100 = 80.0\% \quad \leftarrow \text{계획 생성(하드파싱) CPU} \\ \text{Execute 비중} &= 6.0 \div 40.0 \times 100 = 15.0\% \quad \leftarrow \text{실제 갱신은 소량}\end{aligned}$$

여기서 두 축을 분리해야 합니다.

축 A: 하드파싱(계획 생성·라이브러리 캐시 MISS·재귀 딕셔너리 콜) — 미스 95.0%, 재귀 4배 → 병목

축 B: 파싱 콜 횟수(소프트파싱 · 커서 미보유) — Parse:Execute=1:1이지만 여기서 소프트웨어가 아님

동적 SQL이 리터럴로 텍스트를 매번 바꾸니 커서가 공유되지 않는다. Parse 콜을 캐싱으로 붙잡아도(축 B 처방) 텍스트가 다른 커서는 재사용되지 않는다. 원인은 축 A(하드파싱)이므로 처방도 축 A라야 한다.

처방은 커서 홀드가 아니라 동적 SQL의 리터럴을 바인드 변수로 바꿔 SQL 텍스트를 하나로 고정하는 것입니다. 텍스트가 같아지면 커서가 공유돼 하드파싱이 소프트파싱으로 바뀌고, 딕셔너리 재귀 콜(320만)과 Parse CPU(32초)가 함께 사라집니다. ③이 하드파싱 축(계획 생성·재귀 콜)을 정확히 지목했습니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                                                    |
|-----|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | 재귀 Query 16,000,000·Fetch 1,600,000회는 맞지만 Disk가 0이라는 것은 그 논리 읽기가 이미 100% 캐시 적중이라는 뜻이므로, 버퍼 캐시를 키워 없앨 물리 읽기가 애초에 없고 논리 읽기 횟수도 그대로입니다. 게다가 재귀 statement 자체가 하드파싱이 부른 딕셔너리 조회라 원인이 아니라 결과이고, 하드파싱이 계속되는 한 재귀 콜은 다시 발생합니다 — 뿌리(하드파싱)가 아니라 부산물을 겨냥한 헛짚음입니다 |
| ②   | ×  | Parse:Execute = 1:1과 Parse CPU 32.0초는 맞지만, 이를 소프트파싱 쪽주로 읽은 것이 오류입니다. 라이브러리 캐시 미스가 380,000건(95.0%)이라 이 파싱들은 소프트파싱이 아니라 하드파싱이고, 리터럴로 텍스트가 매번 달라 커서 홀드·session_cached_cursors로 붙잡을 공유 커서 자체가 없습니다. 소프트파싱 빈도 축을 하드파싱 축으로 오인한 별개 축 혼동입니다                   |
| ③   | ○  | 하드파싱율 380,000/400,000 = 95.0%로 계획 생성 축이 병목임을 짚고, 그 하드파싱이 재귀 딕셔너리 콜 320만(비재귀의 4배)을 유발했음을 연결한 뒤, 동적 SQL의 리터럴을 바인드 변수로 바꿔 텍스트를 고정하는 처방이 정확합니다                                                                                                            |
| ④   | ×  | Execute 400,000콜이 Query 800,000·Current 400,000으로 400,000행을 갱신한 것은 맞지만, Execute CPU 5.0초·Elapsed 6.0초는 전체 40초의 15.0%뿐이고 건당 논리 읽기도 2~3블록에 그칩니다. 시간을 지배하는 것은 Parse CPU 32초(80.0%)의 하드파싱 축이므로, 인덱스를 보강해 UPDATE 실행계획을 다시 짜도 파싱 지연은 그대로 남습니다               |

#### ■ 이 문제의 핵심

- 하드파싱율 = 라이브러리 캐시 미스 ÷ Parse 콜. 여기서는 380,000/400,000 = 95.0%로, 동적 SQL이 리터럴을 붙여 텍스트가 매번 달라 커서가 공유되지 못해 사실상 매 실행이 하드파싱입니다.
- 하드파싱은 재귀 Call을 낳습니다. 계획을 새로 만들 때마다 데이터 딕셔너리를 조회하므로 재귀 statement가 320만 콜(비재귀의 4배)로 폭증했습니다 — 재귀 축은 하드파싱의 부산물입니다.
- 하드파싱 축(계획 생성)과 소프트파싱 빈도 축(커서 미보유)은 별개입니다. Parse:Execute = 1:1이어도 미스율이 95.0%면 커서 캐싱은 붙잡을 공유 커서가 없어 무력합니다.
- 처방은 커서 홀드·버퍼 캐시·실행계획 튜닝이 아니라 동적 SQL의 리터럴 → 바인드 변수입니다. 텍스트가 하나로 고정돼야 커서가 공유됩니다.
- Parse CPU 32초(80.0%)가 시간을 지배하고 Execute는 15.0%뿐이라, 병목은 실행이 아니라 파싱(계획 생성)에 있습니다.

■ 한 줄 정리 하드파싱율이 95.0%(380,000/400,000)이고 재귀 딕셔너리 콜이 320만(비재귀의 4배)으로 폭증한 것은 동적 SQL의 리터럴 때문에 커서가 공유되지 못해 매 실행이 하드파싱이기 때문이며, 처방은 커서 홀드·버퍼 캐시가 아니라 리터럴을 바인드 변수로 바꿔 하드파싱을 소프트파싱으로 돌리는 것입니다.

## 문제 4. 정답 ④

응답시간 분석의 출발점은 시간을 성질이 다른 두 축으로 쪼개는 것입니다.

Response Time = Service Time + Wait Time

Service Time : CPU를 점유해 실제 일한 시간 (on CPU)

Wait Time : 자원을 기다리며 CPU를 놓고 멈춘 시간 (off CPU)  
= 여러 대기 이벤트의 합  
(디스크 I/O, 락, 래치, 로그 sync, 네트워크, 사용자 대기 ...)

진단 순서

- CPU 시간과 대기 이벤트별 시간을 분리
- 응답시간을 '지배'하는 축을 확인
- 지배 요인에 맞는 처방 선택

CPU가 지배하면 논리 읽기(buffer get) 축소를, 특정 대기가 지배하면 그 이벤트를 겨냥한 처방을 골라야 합니다. ④는 이 분해와 진단 순서를 옳게 서술했습니다. 핵심은 시간을 먼저 쪼개고 지배 요인을 확인한 뒤 처방을 정하는 것입니다.

①·②·③은 앞머리에 참인 관찰(Elapsed-CPU가 CPU를 놓고 멈춘 시간·CPU 비중이 크면 Wait Time이 작다·유휴 대기도 Elapsed에 더해진다)을 깔지만, 결론에서 병목의 원인을 영뚱한 요인에 오귀속합니다. Elapsed-CPU를 통째로

디스크 대기(①), 높은 CPU를 곧 효율로(②), 사용자 응답 대기를 서버 자원 문제로(③) 돌린 것이 각각의 오류입니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                  |
|-----|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | Elapsed-CPU가 CPU를 놓고 멈춰 있던 시간이라는 앞부분은 맞지만, 그것은 물리 I/O 하나가 아니라 전체 Wait Time이라 디스크 외에 락·래치·로그 sync·네트워크·사용자 대기가 함께 섞입니다. 대기 이벤트별 분해를 건너뛰어 통째로 디스크 대기라고 보고 스토리지를 겨는 것이 원인 오귀속입니다       |
| ②   | ×  | CPU 비중이 크면 Wait Time이 작다는 앞부분은 맞지만, 그 큰 CPU는 효율의 증거가 아니라 불필요한 논리 읽기(buffer get)로 CPU를 태우는 신호일 수 있습니다. 논리 읽기를 확인하지 않고 "CPU 많이 씬=효율적"으로 결론지어 튜닝 대상에서 뺀 것이 병목 원인을 뒤집은 오귀속입니다           |
| ③   | ×  | <b>SQL*Net message from client</b> 가 Wait Time에 집계돼 Elapsed를 늘린다는 앞부분은 맞지만, 이는 클라이언트·애플리케이션 쪽 응답을 기다리는 유휴(idle) 대기라 서버가 일한 시간이 아닙니다. 응답시간에 더해졌다는 이유로 서버 부하로 돌려 자원 증설을 처방한 것이 오귀속입니다 |
| ④   | ○  | 응답시간을 Service Time과 Wait Time으로 분해하고, CPU·대기 이벤트별 시간을 먼저 분리해 지배 요인을 가른 뒤 그에 맞는 처방을 고르는 응답시간 분석의 원칙을 정확히 서술했습니다                                                                      |

#### ▪ 이 문제의 핵심

1. Response Time = Service Time(CPU 점유) + Wait Time(대기)가 응답시간 분석의 기본 등식입니다.
2. Wait Time은 여러 대기 이벤트의 합이라 디스크 하나로 뭉뚱그릴 수 없습니다(락·래치·로그 sync·네트워크·사용자 대기 포함).
3. 진단은 CPU와 대기 이벤트별 시간을 먼저 분리해 응답시간을 지배하는 축을 확인하는 데서 시작합니다.
4. 높은 CPU가 곧 효율은 아니며, **SQL\*Net message from client** 같은 유휴 대기는 서버 자원 증설로 줄지 않습니다.

▪ 한 줄 정리 응답시간을 Service Time과 Wait Time으로 분리해 지배 요인을 먼저 가른 뒤 처방을 고르는 ④가 응답시간 분석의 원칙이고, Elapsed-CPU를 디스크로·높은 CPU를 효율로·사용자 대기를 서버 부하로 돌린 나머지는 병목 원인을 오귀속한 진단입니다.

### 문제 5. 정답 ②

먼저 BCHR로 논리 대비 물리 읽기를 봅니다.

$$\begin{aligned} \text{BCHR} &= ((\text{Query} + \text{Current}) - \text{Disk}) / (\text{Query} + \text{Current}) \times 100 \\ &= (5,000,000 + 0 - 200,000) / 5,000,000 \times 100 \\ &= 4,800,000 / 5,000,000 \times 100 = 96.0\% \end{aligned}$$

$$\text{물리 읽기 비중} = 200,000 / 5,000,000 \times 100 = 4.0\%$$

캐시 적중이 96%로 높습니다. 논리 읽기 500만 중 실제 디스크에서 올라온 것은 20만(4%)뿐입니다. 그런데도 느립니다 — 이때는 물리 I/O가 아니라 논리 읽기 자체의 양을 의심해야 합니다. 시간의 정체를 봅니다.

$$\begin{aligned} \text{CPU 비중} &= 40.000 / 50.000 \times 100 = 80.0\% \quad \rightarrow \text{CPU가 시간을 지배} \\ \text{대기 시간} &= 50.000 - 40.000 = 10.0\text{초 (20\%)} \quad \leftarrow \text{그중 물리 읽기 대기} \end{aligned}$$

응답의 80%가 CPU입니다. 물리 읽기가 작는데 CPU가 큰 것은, 캐시에 있는 블록을 반복해서 논리적으로 읽느라(buffer get) CPU를 태운다는 신호입니다. 어디서 났는지 Row Source로 찾되, cr(논리)과 pr(물리)을 분리합니다.

pw(temp spill) = 모든 노드 0 → spill 없음  
 논리 읽기(cr) 정체 = NL 조인의 해양심층수취수량 반복 방문 = 인덱스 재탐색 3,200,000 + 테이블 랜덤 self 1,799,975  
 (테이블 self = TABLE ACCESS BY INDEX ROWID 4,999,975 - 자식 INDEX 3,200,000 = 1,799,975)  
 → NL 조인이 드라이빙 건마다 (인덱스+테이블) 방문을 반복해 200만 행마다 블록을 논리적으로 읽음 → cr 500만  
 물리 읽기(pr) = 200,000 (해양심층수취수량), 그 노드 time의 대부분은 물리 대기가 아니라 논리 읽기 CPU

**pw=0**이라 spill은 없고, **pr**은 20만으로 작습니다. 정체는 NL 조인이 **해양심층수취수량**을 200만 번 방문하며 인덱스 재탐색(cr 3,200,000)과 테이블 랜덤 액세스(self 1,799,975)로 쌓은 논리 읽기 500만이고, 그 대부분이 캐시 적중이라 디스크가 아닌 CPU를 태웁니다.

BCHR 96% 높음 → 물리 디스크는 병목이 아님(Disk 4%)  
 CPU 80% 지배 → 논리 읽기(buffer get) 500만이 CPU를 소진 → 논리 I/O 바운드

따라서 처방은 물리 읽기가 아니라 논리 읽기 횟수 자체를 줄이는 것 — NL 조인을 해시 조인으로 바꾸거나 액세스 경로를 개선해 **해양심층수취수량** 방문 횟수를 낮추는 것입니다. ②가 BCHR과 병목 지목을 함께 옳게 했습니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                                                                 |
|-----|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | Disk 200,000이 테이블 노드(pr=200000)에 몰려 있고 인덱스 노드가 pr=0인 것은 맞지만, 그 200,000은 논리 읽기 500만의 4%뿐이라 BCHR이 96%입니다. 테이블 노드의 time 40,000,000us도 물리 대기가 아니라 cr 4,999,975를 훑는 논리 읽기 CPU가 대부분이므로, 버퍼 캐시를 키워 4%를 없애도 CPU 80%를 만드는 논리 읽기 500만은 그대로입니다 — 논리 I/O 병목을 물리 I/O로 오귀속한 것입니다 |
| ②   | ○  | BCHR (5,000,000-200,000)/5,000,000 = 96.0%로 캐시 적중이 높아 물리 읽기가 4%뿐임을 짚고, CPU 80%·pw=0·cr 500만이 <b>해양심층수취수량</b> 반복 방문에 쌓였음을 연결해 병목을 논리 읽기(buffer get)로 지목하고 논리 읽기 축소를 처방한 것이 정확합니다                                                                                    |
| ③   | ×  | Elapsed 50.000 - CPU 40.000 = 10초와 Fetch 2,001회는 맞지만, 그 10초는 전체의 20%뿐이고 시간의 80%는 CPU입니다. 게다가 2,001회는 배열 인출이 1,000행씩 정상 동작한 결과일 뿐 대기의 반복 누적이 아닙니다. 20%짜리 10초를 I/O 바운드 근거로 삼아 DOP를 올리자는 것은 CPU를 태우는 논리 읽기 병목을 물리 대기로 오귀속한 진단입니다                                      |
| ④   | ×  | NESTED LOOPS의 time 49,000,000us와 2,000,000행은 맞지만, 모든 노드가 pw=0이라 temp로 흘린 spill은 없습니다. pr=200000은 temp 되읽기가 아니라 <b>해양심층수취수량</b> 블록의 물리 읽기이고, NL 조인에는 대량 해시·정렬 영역이 개입하지 않으므로 PGA를 키워도 50초는 그대로입니다 — 논리 읽기 CPU를 집계 spill로 오귀속한 것입니다                                   |

#### ▪ 이 문제의 핵심

1. BCHR = ((Query+Current)-Disk)/(Query+Current) × 100. 여기서는 (5,000,000-200,000)/5,000,000 = 96.0%로 캐시 적중이 높아, 논리 읽기의 4%만 디스크에서 올라왔습니다.
  2. BCHR이 높음에도 느리면 논리 읽기 양을 의심합니다. 물리 I/O가 작는데 CPU가 80%면, 캐시에 있는 블록을 반복 논리 읽기(buffer get)하며 CPU를 태우는 논리 I/O 바운드입니다.
  3. cr과 pr·pw를 분리해야 합니다. cr 500만은 NL 조인이 **해양심층수취수량**을 200만 번 방문해 쌓은 것이고, pr은 20만·pw는 0입니다.
  4. 높은 BCHR이 좋은 신호만은 아닙니다. 불필요한 논리 읽기가 많아도 캐시에서 나오면 BCHR은 높게 뜨므로, 병목은 물리가 아니라 논리 읽기 횟수일 수 있습니다.
  5. 처방은 논리 읽기 축소(NL→해시 조인 전환·액세스 경로 개선)입니다. 버퍼 캐시·DOP·PGA는 CPU를 태우는 논리 I/O를 근거부터 잘못 겨냥합니다.
- 한 줄 정리 BCHR 96%로 캐시 적중이 높아 물리 읽기가 4%뿐이고 pw=0이라 spill도 없는데 CPU가 80%인 것은 NL 조인이 **해양심층수취수량**을 200만 번 방문해 논리 읽기 500만을 쌓았기 때문이므로 병목은 논리 I/O(buffer get)이고, 이를 물리 읽기·DOP·집계 spill로 돌리면 원인을 오귀속하게 된다.

### 문제 6. 정답 ②

플랜의 두 노드에서 나온 Card 값을 나란히 놓습니다.

INDEX RANGE SCAN 요양\_IX Card 8,400 ← 인덱스에서 훑고 남은 건수  
TABLE ACCESS BY INDEX ROWID 산재요양급여 Card 2,100 ← 테이블까지 거친 뒤 남은 건수

**요양\_IX**는 (의료기관코드, 요양개시일자) 순서입니다. WHERE에는 **의료기관코드** = 'M071'(등치)과 **요양개시일자** 범위가 있으므로, 선두 칼럼 등치 + 두 번째 칼럼 범위까지가 인덱스로 연속해서 훑을 수 있는 구간입니다. 그 결과가 INDEX RANGE SCAN의 8,400건입니다.

요양구분은 **요양\_IX**의 구성 칼럼이 아닙니다. 인덱스에 값이 없으니 인덱스 단계에서는 걸러낼 수 없고, ROWID로 테이블 블록을 읽어 온 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 8,400건이 이 단계를 거치며 2,100건으로 줄어듭니다.

8,400 (인덱스 액세스: 의료기관코드=, 요양개시일자 범위)  
└─ 테이블 필터(요양구분='입원') 적용 → 2,100 (최종)

즉 ②가 이 구조를 정확히 서술합니다. 요양구분까지 인덱스 액세스 조건으로 끌어올린 ①, 선두 칼럼 등치까지 필터로 밀어 넣은 ③, 인덱스 반환 건수를 최종 건수로 본 ④는 이 Card 흐름과 어긋납니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                                                           |
|-----|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | Id 3의 Cost 26·Card 8,400은 맞지만, 요양구분은 <b>요양_IX</b> 구성 칼럼이 아니므로 인덱스 액세스 조건이 될 수 없습니다. 세 조건이 다 스캔 범위를 좁혔다면 Id 3의 Card가 이미 2,100이어야 하는데 실재는 8,400이고, Id 2의 Card 2,100도 조인 감소분이 아니라 요양구분 필터가 걸려 낸 결과입니다(의료기관_PK는 1:1이라 건수를 줄이지 못함) — 부분(필터)을 전체(액세스 조건)에 끼운 진술입니다 |
| ②   | ○  | 의료기관코드(등치)·요양개시일자(범위)는 인덱스 액세스 조건, 요양구분은 인덱스에 없어 TABLE ACCESS 단계 필터 — Card가 8,400→2,100으로 줄어드는 흐름과 일치합니다                                                                                                                                                     |
| ③   | ×  | 요양개시일자가 범위 조건이라는 앞부분은 맞지만, 그 성질을 선두 칼럼에까지 확대한 진술입니다. 선두 칼럼 등치는 그 자체로 액세스 조건이고 범위는 그 다음 칼럼부터 적용됩니다. 등치까지 필터로 밀렀다면 620만 건 전체를 훑어야 해 Id 3의 Cost가 26에 그칠 수 없고, INDEX RANGE SCAN 자체가 성립하지 않습니다                                                                    |
| ④   | ×  | Id 3의 Card 8,400은 인덱스 단계 건수일 뿐이고, 요양구분 필터를 거친 Id 2의 Card 2,100이 최종 건수입니다. Bytes 94,500도 비용 표현이 아니라 2,100건 × 45바이트로, Card 2,100이 실제 행 수임을 뒷받침합니다 — 테이블 액세스 단계에서 건수 감소가 분명히 일어납니다                                                                              |

#### ■ 이 문제의 핵심

1. 결합 인덱스는 선두 칼럼 등치 + 다음 칼럼 범위까지가 액세스 조건. 여기서는 의료기관코드(=)·요양개시일자(범위)가 INDEX RANGE SCAN 구간을 만듭니다(Card 8,400).
2. 인덱스에 없는 칼럼은 테이블 필터. 요양구분은 ROWID로 테이블을 읽은 뒤 TABLE ACCESS BY INDEX ROWID 단계에서 걸러집니다.
3. 두 노드의 Card 차이(8,400 vs 2,100)가 곧 테이블 필터의 효과입니다. 인덱스 반환 건수 ≠ 최종 건수.
4. 테이블 필터 조건을 인덱스 액세스 조건 집합에 끼워 넣지 않습니다. 인덱스에 없는 요양구분은 스캔 범위를 정하는 데 관여하지 못합니다.

■ 한 줄 정리 선두 칼럼 등치(의료기관코드)와 다음 칼럼 범위(요양개시일자)는 인덱스 액세스 조건이 되고, 인덱스에 없는 요양구분은 테이블 액세스 단계 필터가 되어 Card가 8,400에서 2,100으로 줄어든다.

### 문제 7. 정답 ①

세 지표를 차례로 계산해 두 가설("왕복이 병목"·"손익분기점을 넘었으니 풀 스캔")을 각각 검증한 뒤 병목을 종합합니다.

① 배열 크기(Array Processing). 마지막 한 번은 "더 없음(no-more-data)" 인출이라 -1 합니다.

$$\begin{aligned} \text{ArraySize} &= \text{총 Rows} \div (\text{Fetch 횟수} - 1) \\ &= 300,000 \div (376 - 1) \\ &= 300,000 \div 375 = 800 \text{ (행/Fetch)} \end{aligned}$$

→ 한 번에 800행씩 담아 Fetch는 376회뿐. 배열 인출은 이미 총분 → 왕복 가설 기각

② 손익분기점(인덱스 경유 vs 풀 스캔). 인덱스 cr은 테이블 노드에 누적되므로 두 경로의 블록 수를 견칩니다.

$$\begin{aligned} \text{인덱스 경유 블록} &= \text{INDEX cr } 1,000 + \text{테이블 랜덤 블록 cr } 60,000 = 61,000 \\ \text{풀 스캔 블록} &= \text{테이블 총 블록} \approx 400,000 \\ &\rightarrow 61,000 < 400,000 : \text{손익분기점 미도달} \rightarrow \text{인덱스가 여전히 유리(풀 스캔 전환은 손해)} \\ \text{손익분기점 도달 행수} &= \text{풀 스캔 블록} \div \text{CF 밀도} = 400,000 \div 0.20 = 2,000,000\text{행} \\ &\rightarrow \text{읽는 } 300,000\text{행은 } 2,000,000\text{행에 한참 못 미쳐 아직 인덱스 우위 구간} \end{aligned}$$

③ 클러스터링 팩터(CF). 인덱스 cr을 뺀 테이블 랜덤 블록만 떼어 냅니다.

$$\begin{aligned} \text{테이블 랜덤 블록(cr)} &= \text{TABLE 노드 cr} - \text{INDEX 노드 cr} \\ &= 61,000 - 1,000 = 60,000 \\ \text{CF 밀도} &= \text{테이블 랜덤 블록(cr)} \div \text{ROWID 수} \\ &= 60,000 \div 300,000 = 0.20 \\ \text{완전 정렬 시 밀도} &= \text{테이블 총 블록} \div \text{테이블 총 행} \\ &= 400,000 \div 80,000,000 = 0.005 \quad (\text{블록당 } 200\text{행}) \\ \text{약화 배수} &= 0.20 \div 0.005 = 40\text{배} \end{aligned}$$

CF 밀도 0.20은 완전 정렬값(0.005)의 40배 — 인덱스로 뽑은 행들이 저마다 다른 테이블 블록에 흩어져 있어, 300,000행을 읽으며 60,000번의 테이블 랜덤 블록 액세스가 일어났다는 뜻입니다. 완전히 정렬됐다면 블록당 200행씩 담겨 약 1,500블록이면 될 것을, 60,000블록을 방문한 셈입니다. 세 지표를 종합합니다.

$$\begin{aligned} \text{ArraySize } 800 &\rightarrow \text{배열 인출 정상, Fetch } 376\text{회} \rightarrow \text{왕복 병목 아님(가설 1 기각)} \\ \text{손익분기점} &\rightarrow \text{인덱스 } 61,000 < \text{풀 스캔 } 400,000 \rightarrow \text{미도달} \rightarrow \text{풀 스캔 전환은 손해(가설 2 기각)} \\ \text{CF 밀도 } 0.20 &\rightarrow \text{인덱스로 뽑은 행마다 다른 블록} \rightarrow \text{테이블 랜덤 액세스 } 60,000\text{회} \rightarrow \text{병목의 뿌리} \end{aligned}$$

시간·논리 읽기도 이 랜덤 액세스에 몰립니다.

$$\begin{aligned} \text{INDEX RANGE SCAN cr} &= 1,000 \text{ (전체 cr } 61,000\text{의 } 1.6\%) && \leftarrow \text{인덱스는 가벼움} \\ \text{TABLE 랜덤 액세스 cr} &= 60,000 \text{ (98.4\%)} && \leftarrow \text{논리 읽기의 대부분} \end{aligned}$$

TABLE 랜덤 액세스 time = 49초 (전체 50초의 약 98%) ← 시간의 대부분  
CPU 비중 = 8.0 / 50.0 × 100 = 16.0% → 대기 바운드

세 지표가 함께 말해 줍니다: 배열 크기(800)는 정상이라 왕복이 아니고, 손익분기점 미도달이라 풀 스캔은 손해이며, CF 밀도(0.20, 완전 정렬의 40배)가 만든 테이블 랜덤 액세스가 병목입니다. 처방은 배열 크기 상향도 풀 스캔 전환도 아니라, 필요한 컬럼을 인덱스에 담아 커버링으로 테이블 액세스를 없애거나 인덱스 키 순서로 테이블을 재정렬(CF 개선)하는 것입니다. ①이 세 지표를 옳게 결합했습니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                             |
|-----|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ○  | ArraySize 300,000/(376-1) = 800으로 왕복 가설을, 인덱스 61,000 < 풀 스캔 400,000(손익분기점 미도달)으로 풀 스캔 가설을 각각 기각하고, CF 밀도 60,000/300,000 = 0.20(완전 정렬 0.005의 40배)으로 테이블 랜덤 액세스를 병목으로 지목한 뒤 커버링·CF 개선을 처방한 것이 정확합니다                              |
| ②   | ×  | pr 6,000·pr 100과 테이블 노드 time 49,000,000us는 맞지만, pr 6,000은 논리 읽기 61,000의 10%뿐이고 그 물리 읽기조차 나쁜 CF가 흠뻑된 블록을 읽다 캐시에 없던 뉘입니다. 버퍼 캐시를 키워 6,000을 캐시 적중으로 돌려도 CF 밀도 0.20이 만든 60,000번의 논리적 테이블 랜덤 방문은 그대로 남으므로, 뿌리(CF)를 물리 읽기로 오귀속한 것입니다 |
| ③   | ×  | Fetch 376회와 CPU 8.0초 대 Elapsed 50.0초는 맞지만, ArraySize = 300,000/(376-1) = 800으로 배열 크기가 이미 커 왕복은 376회뿐입니다. 42초 격차는 왕복 대기가 아니라 테이블 노드 time 49초에 잡힌 랜덤 블록 액세스이므로, 배열 크기를 더 키워도 60,000번의 랜덤 액세스는 그대로 남습니다 — 개발자 가설을 그대로 따른 오귀속입니다   |
| ④   | ×  | 테이블 랜덤 블록이 61,000-1,000 = 60,000이라는 앞부분은 맞지만, 인덱스 경우 총 블록 61,000은 풀 스캔 400,000보다 훨씬 적어 손익분기점(약 2,000,000행)을 넘지 않았습니다. 풀 스캔으로 바꾸면 오히려 400,000블록을 읽어 손해이므로, 병목(나쁜 CF의 테이블 랜덤 액세스)을 손익분기점 초과로 오귀속한 것입니다                           |

#### ■ 이 문제의 핵심

- 세 지표로 두 가설을 각각 기각한 뒤 병목을 종합합니다. ArraySize 800은 왕복 가설을, 손익분기점 미도달(61,000<400,000)은 풀 스캔 가설을 기각하고, CF 밀도 0.20이 진짜 병목을 지목합니다.
- ArraySize = 총 Rows ÷ (Fetch 횟수 - 1). 300,000/(376-1) = 800으로 배열 크기가 이미 커 왕복(376회)은 병목이 아닙니다 — "Fetch가 많다"는 인상만으로 왕복을 탓하면 안 됩니다.
- 손익분기점은 블록 수로 견칩니다. 인덱스 경우 61,000블록이 풀 스캔 400,000블록보다 적어 미도달이므로, 풀 스캔 전환은 손해입니다(손익분기 행수 ≈ 400,000/0.20 = 2,000,000행).
- CF 밀도 = 테이블 랜덤 블록(cr) ÷ ROWID 수. 테이블 cr은 인덱스 cr을 뺀 60,000이고, 60,000/300,000 = 0.20으로 완전 정렬값(400,000/80,000,000 = 0.005)의 40배입니다. 시간(49초)·논리 읽기(98.4%)가 이 노드에 몰립니다.
- 처방은 커버링 인덱스 또는 인덱스 키 순서 재정렬(CF 개선)입니다. 배열 크기·버퍼 캐시·풀 스캔 전환은 60,000번의 랜덤 블록 방문을 없애지 못합니다.

■ 한 줄 정리 ArraySize 800으로 왕복 가설을, 손익분기점 미도달(61,000<400,000)로 풀 스캔 가설을 기각하고 CF 밀도 0.20(60,000/300,000, 완전 정렬의 40배)으로 테이블 랜덤 액세스를 병목으로 지목해야 하며, 배열 크기·버퍼 캐시·풀 스캔 전환은 병목을 잘못 짚은 처방이다.

## 문제 8. 정답 ①

컬럼 순서를 정하는 기준이 하나뿐인지 여럿인지가 갈림길입니다. 선택도는 그중 하나일 뿐입니다.

결합 인덱스 컬럼 순서 결정 - 여러 기준의 우선순위

- (1) 조건절에 '=' (등치)로 자주 쓰이는 컬럼을 앞에 ← 스캔 범위를 좁힘
- (2) '=' 조건 컬럼을 범위(부등호·BETWEEN) 컬럼보다 앞에
- (3) 그다음에 선택도·정렬(ORDER BY)·전체 SQL 절총 고려 ← 선택도는 여기 한 요소

컬럼 순서의 1차 기준은 어떤 컬럼이 = 조건으로 들어와 스캔 범위를 좁히느냐입니다. 선택도(값의 종류 수)는 그 뒤에 고려하는 여러 요소 중 하나이지, 순서를 홀로 결정하는 기준이 아닙니다. 예컨대 선택도가 낮아도 =로 항상 들어오는 컬럼을 앞에 두는 편이, 선택도만 높고 범위조건으로 쓰이는 컬럼을 앞세우는 것보다 유리할 때가 많습니다.

①은 이 하위 기준(선택도)을 전체 등식으로 부풀렸습니다. "선택도 높은 컬럼을 앞에 두는 것이 원칙이고 이 한 가지로 최적 순서가 정해진다"고 했지만, 선택도는 순서 결정에 관여하는 여러 요소 중 하나일 뿐이며 등치/범위 조건 형태·정렬·전체 SQL 절총이 함께 작용합니다. 부분 기준을 유일한 전체 기준으로 격상한 서술입니다.

②·③·④는 옳습니다. 커버링용 컬럼 추가의 효용과 비용을 함께 짚은 점(②), = 조건 컬럼을 범위 컬럼보다 앞세워야 스캔 범위가 좁아진다는 실제 1차 기준(③), 여러 SQL을 절총해 하나의 인덱스를 설계한다는 점(④)이 모두 인덱스 설계 원칙에 부합합니다.

| 선택지 | 판정 | 이유                                                                                                                     |
|-----|----|------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | 선택도는 컬럼 순서를 정하는 여러 기준 중 하나일 뿐입니다. 1차 기준은 = 조건으로 스캔 범위를 좁히는 컬럼을 앞세우는 것이며, 선택도 하나로 순서가 결정된다는 것은 하위 기준을 전체 원칙으로 부풀린 서술입니다 |
| ②   | ○  | 참조 컬럼을 뒤에 덧붙여 커버링하면 테이블 랜덤 액세스를 없앨 수 있으나, 인덱스 크기·DML 부하가 늘므로 필요한 경우로 한정해야 한다는 균형이 맞습니다                                 |
| ③   | ○  | = 조건 컬럼을 범위 조건 컬럼보다 앞에 두면 인덱스 스캔 시작·종료 지점이 좁혀져, 리프를 훑다 필터로 버리는 행이 줄어드는 실제 1차 기준입니다                                     |
| ④   | ○  | 하나의 결합 인덱스로 여러 SQL을 충족하려면 개별 최적이지 아니라 전체 SQL 집합의 액세스 패턴·빈도를 함께 보고 절충해야 한다는 설계 원칙이 옳습니다                                 |

#### ■ 이 문제의 핵심

- 결합 인덱스 컬럼 순서는 여러 기준의 우선순위로 정해지며, 선택도는 그중 하나일 뿐입니다.
- 1차 기준은 =(등치) 조건으로 스캔 범위를 좁히는 컬럼을 앞세우는 것이고, = 컬럼을 범위 컬럼보다 앞에 둡니다.
- 커버링용 컬럼 추가는 테이블 랜덤 액세스를 줄이지만 인덱스 크기·DML 부하를 늘리므로 남용을 피합니다.
- 하나의 인덱스로 여러 SQL을 충족하려면 전체 SQL 집합을 절충해 설계합니다.

■ 한 줄 정리 선택도는 컬럼 순서를 정하는 여러 요소 중 하나일 뿐이고 1차 기준은 = 조건으로 스캔 범위를 좁히는 컬럼인데, "선택도 한 가지로 최적 순서가 정해진다"고 한 ①은 하위 기준을 유일한 전체 원칙으로 부풀린 서술입니다.

### 문제 9. 정답 ②

NL 조인은 드라이빙 한 건마다 내부를 다시 탐색하는 반복 구조라, 어디에 인덱스가 있어야 하고 어떤 상황에 유리한지가 정해집니다.

NL 조인의 반복 구조

```
for 드라이빙 테이블의 조건 만족 행 r:      ← 바깥(outer) 루프
    내부 테이블을 r의 조인 값으로 탐색      ← 안쪽(inner), 매 건 반복
```

유리한 조건

- 내부 테이블 조인 컬럼에 인덱스 → 개별 탐색이 빠름 (반복의 단가↓)
- 드라이빙 결과 집합이 작음 → 반복 횟수↓
- 한 건씩 만들어 흘리므로 부분범위처리(Top-N·먼저 나온 일부)에 강함

반복 횟수는 드라이빙 결과 건수가, 반복 한 번의 비용은 내부 테이블 탐색 방식이 좌우합니다. 따라서 내부 조인 컬럼에 인덱스가 있고 드라이빙 결과가 크지 않을 때 NL 조인이 유리합니다. ②는 이 구조와 유리 조건을 옳게 서술했습니다. 이 원리는 오라클·SQL Server에 공통입니다.

①·③·④는 앞머리에 참인 구조 설명(개별 탐색 단가가 낮다·드라이빙 행을 하나씩 읽어 내부를 반복 탐색한다·반복은 바깥 루프에서 시작한다)을 깔고도, 뒷문장에서 키워드에서 온 반사적 연상을 결론으로 굳혀 틀렸습니다. "인덱스 반복이니 대량에도 빠르다"(①), "조인이니 전부 만든 뒤 반환한다"(③), "성능은 드라이빙 인덱스에 달렸다"(④)는 각각 그럴듯한 연상이지만, NL은 대량·전건 조인에 오히려 불리하고, 한 건씩 흘려 부분범위처리에 강하며, 정작 인덱스가 중요한 쪽은 반복 탐색되는 내부 테이블입니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                            |
|-----|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | 개별 탐색 단가가 낮고 해서 테이블 준비 비용이 없다는 앞부분은 맞지만, 드라이빙이 수백만 건이면 그 낮은 단가에 반복 횟수가 곱해져 총비용이 폭증합니다. 해서 조인은 준비 비용을 한 번만 치르고 대량을 훑으므로 그 구간에서 유리합니다 — "인덱스 반복=대량에도 빠름"은 단가만 보고 반복 횟수를 지운 반사적 연상입니다            |
| ②   | ○  | 드라이빙 한 건마다 내부를 반복 탐색하는 구조를 짚고, 내부 조인 컬럼 인덱스와 작은 드라이빙 결과 집합이라는 유리 조건을 정확히 서술했습니다                                                                                                               |
| ③   | ×  | 드라이빙 행을 하나씩 읽어 내부를 반복 탐색한다는 앞부분은 맞지만, 바로 그 구조 덕에 NL 조인은 한 건이 짝을 찾는 즉시 결과를 흘려보냅니다. 반복을 끝까지 돌릴 필요가 없어 오히려 부분범위처리(Top-N·먼저 나온 일부만 반환)에 강하고, 조건이 맞으면 전건을 만들기 전에 멈춥니다 — "조인=전부 만든 뒤 반환"은 반사적 연상입니다 |
| ④   | ×  | 반복이 바깥 루프에서 시작한다는 앞부분은 맞지만, 되풀이해 탐색당하는 쪽은 내부 테이블이므로 성능을 좌우하는 인덱스도 내부 조인 컬럼 쪽입니다. 드라이빙 건수를 줄여 놓아도 내부를 건마다 Full Scan하면 그 건수만큼 전건 스캔이 곱해져 부담이 폭증합니다 — "드라이빙 인덱스가 관건"은 초점을 뒤바꾼 반사적 연상입니다          |

#### ■ 이 문제의 핵심

- NL 조인은 드라이빙 한 건마다 내부 테이블을 반복 탐색하는 구조입니다.

- 반복 횟수는 드라이빙 결과 건수가, 반복 한 번의 비용은 내부 테이블 탐색 방식이 좌우합니다.
- 성능을 좌우하는 인덱스는 반복 탐색되는 내부 조인 컬럼 쪽이며, 내부 Full Scan 반복은 부담이 폭증합니다.
- NL은 한 건씩 만들어 흘리므로 부분범위처리(Top-N)에 강하고, 대량 전건 조인에는 해시 조인이 유리합니다.

▪ 한 줄 정리 NL 조인은 드라이빙 한 건마다 내부를 반복 탐색하므로 내부 조인 컬럼 인덱스와 작은 드라이빙 결과가 유리 조건인데, 이를 옮겨 짚은 ②가 정답이고 "대량에도 빠른·전건 후 반환·드라이빙 인덱스 관련"은 키워드에서 온 반사적 연상입니다.

## 문제 10. 정답 ②

해시 조인 플랜에서 첫 번째(위) 자식이 Build Input이고 두 번째(아래) 자식이 Probe Input입니다. 이 플랜은 Id 2 수취은행이 첫 자식이므로 Build=수취은행, Probe=해외송금거래입니다. 이 방향을 결정하는 규칙 두 가지를 봅니다.

규칙 1) LEADING의 선행(첫) 테이블이 기본적으로 Build Input이 된다.

규칙 2) SWAP\_JOIN\_INPUTS(x)는 x를 Build로 강제(기본 방향을 뒤집음),

NO\_SWAP\_JOIN\_INPUTS(x)는 x가 Build로 스왑되지 않게(Probe 유지) 한다.

각 선택지가 만들어 내는 Build 방향을 짚습니다.

- |                                                   |                             |        |
|---------------------------------------------------|-----------------------------|--------|
| ① LEADING(b t) USE_HASH(t)                        | → 선행 b(수취은행)=Build          | 플랜과 일치 |
| ② LEADING(t b) USE_HASH(b)                        | → 선행 t(해외송금거래)=Build, 스왑 없음 | 방향 반대  |
| ③ LEADING(t b) USE_HASH(b) SWAP_JOIN_INPUTS(b)    | → 선행 t=Build이나 스왑으로 b=Build | 일치     |
| ④ LEADING(b t) USE_HASH(t) NO_SWAP_JOIN_INPUTS(t) | → 선행 b=Build, t는 Probe 고정   | 일치     |

②는 **LEADING(t b)**로 대형 해외송금거래를 선행에 두고 스왑 힌트가 없으므로 Build=해외송금거래(대형)가 되어, 플랜의 Build=수취은행(Id 2)과 Build/Probe가 뒤바뀝니다. 나머지 ①③④는 선행 지정이나 스왑으로 Build가 소형 수취은행이 되어 플랜과 같습니다. 따라서 이 플랜을 낼 수 없는, 가장 적절하지 않은 힌트는 ②입니다.

플랜: Build=수취은행(Id 2, 소형) / Probe=해외송금거래(Id 3, 대형)

②: Build=해외송금거래(대형) / Probe=수취은행(소형) ← 두 입력을 맞바꾼 값스왑

| 선택지 | 판정 | 이유                                                                                                                                           |
|-----|----|----------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ○  | <b>LEADING(b t)</b> 가 소형 수취은행 b를 선행에 두어 기본적으로 b가 Build가 되므로, Id 2(수취은행)=Build인 플랜을 그대로 냅니다                                                   |
| ②   | ×  | <b>LEADING(t b)</b> 로 대형 해외송금거래를 선행에 두고 SWAP_JOIN_INPUTS도 없어 Build=해외송금거래가 됩니다. 플랜은 Build=수취은행(Id 2)이므로 Build와 Probe를 맞바꾼 방향이라 이 플랜을 내지 못합니다 |
| ③   | ○  | <b>LEADING(t b)</b> 면 기본 Build는 t지만 <b>SWAP_JOIN_INPUTS(b)</b> 가 b(수취은행)를 Build로 강제하므로, 결과적으로 Id 2=수취은행=Build인 플랜과 일치합니다                     |
| ④   | ○  | <b>LEADING(b t)</b> 로 b=Build가 기본이고 <b>NO_SWAP_JOIN_INPUTS(t)</b> 가 t를 Probe로 고정하므로, Build=수취은행·Probe=해외송금거래인 플랜을 그대로 냅니다                    |

### ▪ 이 문제의 핵심

- 해시 조인 플랜의 첫 자식 = Build Input, 둘째 자식 = Probe Input. 이 플랜은 Id 2(수취은행)가 Build, Id 3(해외송금거래)가 Probe입니다.
- LEADING의 선행 테이블이 기본 Build입니다. 방향을 뒤집으려면 **SWAP\_JOIN\_INPUTS**가 필요합니다.
- ②는 대형을 선행에 두고 스왑이 없어 Build=대형 — 플랜(Build=소형)과 정반대라 이 플랜을 낼 수 없습니다.
- ③·④는 스왑·비스왑 힌트로 Build를 소형으로 맞춘 서로 다른 표현이라 모두 같은 플랜을 냅니다. Build/Probe를 맞바꾼 값스왑만 골라내야 합니다.

▪ 한 줄 정리 플랜의 첫 자식 Id 2가 수취은행이므로 Build=수취은행(소형)인데, ②는 **LEADING(t b)**로 대형 해외송금거래를 선행에 두고 스왑 힌트도 없어 Build=대형이 되어 Build/Probe가 뒤바뀌므로, 이 플랜을 내지 못하는 가장 적절하지 않은 힌트다.

## 문제 11. 정답 ④

서브쿼리 Unnesting은 중첩 서브쿼리를 조인으로 승격시키는 비용 기반 변환입니다. 두 가지를 갈라 봐야 합니다.

① Unnesting 여부 : 조인으로 풀지 말지 → 옵티마이저가 비용으로 판단 (무익하면 FILTER로 잔존)

② build/probe 배치 : 세미 조인으로 풀린 뒤 어느 쪽을 해시 테이블(build)로 둘지 → 역시 비용으로 선택

· 바깥 집합을 build → HASH JOIN SEMI

· 서브쿼리를 build → HASH JOIN RIGHT SEMI

세미 조인이라고 해서 서브쿼리 쪽이 후행(probe)으로 고정되는 것이 아닙니다. 옵티마이저는 두 집합의 크기·통계를 건져 더 작은 쪽을 해시 테이블(build)로 삼는 편이 유리하면 서브쿼리 집합을 build로 올립니다. 그것이 **HASH JOIN RIGHT**

**SEMI**입니다. ④는 Unnesting이 비용 기반이라는 점과, 세미 조인 뒤 build 배치도 비용으로 갈린다는 점을 정확히 짚었습니다.

나머지는 모두 키워드에서 결론으로 곧장 건너뛴 반사적 연상입니다.

①: '서브쿼리 = 확인 대상 = 후행'이라는 반사입니다. 해시 조인이 build를 먼저 올린 뒤 probe를 흘려보낸다는 앞부분은 맞지만, 세미 조인은 build 쪽을 다 소진하지 않고 첫 짝에서 멈추는 조인이라 어느 쪽을 build로 두어도 '존재 확인'이 성립합니다. 서브쿼리 쪽을 build로 삼는 **HASH JOIN RIGHT SEMI**가 실제로 존재하므로, 그 형태가 나오지 않는다는 단정은 틀립니다.

②: '**IN** = 중복 없앤 인라인 뷰 + 일반 조인'이라는 반사입니다. 중복 값이 바깥 행을 증식시킨다는 앞부분은 맞지만, 그 증식을 막는 방식이 다릅니다. **IN** 서브쿼리의 Unnesting 결과는 세미 조인이고, 세미 조인은 서브쿼리를 미리 유일화하지 않은 채 조인 단계에서 첫 짝만 취해 증식을 막습니다.

③: 'Unnesting = 반복 제거 = 이득이니 무조건'이라는 반사입니다. 풀지 않으면 FILTER로 반복 수행된다는 앞부분은 맞지만, 서브쿼리 결과가 작고 캐싱이 잘 들으면 그 FILTER가 더 쌀 수 있습니다. Unnesting은 비용 기반이라 변환이 불리하다고 평가되면 조인으로 풀지 않고 FILTER로 남깁니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                              |
|-----|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | 해시 조인이 build를 먼저 올린 뒤 probe를 흘려보낸다는 앞부분은 맞지만, 세미 조인은 첫 짝에서 멈추므로 어느 쪽을 build로 두어도 존재 확인이 성립하고 서브쿼리 쪽을 build로 삼는 <b>HASH JOIN RIGHT SEMI</b> 가 실제로 존재합니다. '서브쿼리=확인 대상=후행'이라는 반사로 build 배치가 비용으로 갈린다는 사실을 지운 진술입니다 |
| ②   | ×  | 서브쿼리 쪽 중복 값이 바깥 행을 증식시킨다는 앞부분은 맞지만, <b>IN</b> Unnesting의 결과는 세미 조인이라 서브쿼리를 미리 유일화하지 않은 채 조인 단계에서 첫 짝만 취해 증식을 막습니다. ' <b>IN</b> =유일화 인라인 뷰+일반 조인'은 세미 조인을 일반 조인으로 바꿔치기한 반사적 연상입니다                                |
| ③   | ×  | 풀지 않으면 바깥 행마다 FILTER로 되풀이된다는 앞부분은 맞지만, 서브쿼리 결과가 작고 캐싱이 잘 들으면 그 FILTER가 더 쌀 수 있어 Unnesting은 비용 기반으로 판단되고 불리하면 FILTER로 남습니다. '반복 제거니 이득=무조건 변환'은 비용 판단을 건너뛴 반사적 연상입니다                                             |
| ④   | ○  | Unnesting은 비용 기반 변환이고, 세미 조인으로 풀린 뒤 build 배치도 비용으로 갈려 서브쿼리 쪽이 build가 되는 <b>HASH JOIN RIGHT SEMI</b> 로도 수행될 수 있습니다                                                                                               |

#### ▪ 이 문제의 핵심

1. 서브쿼리 Unnesting은 비용 기반 변환. 조인으로 푸는 편이 유리할 때 수행되고, 무익하면 서브쿼리는 FILTER로 남습니다.

2. 세미 조인이라고 서브쿼리가 후행 고정은 아니다. 옵티마이저는 비용으로 build/probe를 배치하며, 서브쿼리를 build로 올리면 **HASH JOIN RIGHT SEMI**가 됩니다.

3. **IN** Unnesting = 세미 조인. 조인 단계에서 중복 증식을 막을 뿐, 서브쿼리를 먼저 유일화해 일반 조인하는 것과 같지 않습니다.

4. '반복 제거니 무조건 이득'은 반사적 연상. 변환 여부도, build 배치도 모두 비용으로 판단됩니다.

▪ 한 줄 정리 서브쿼리 Unnesting은 비용 기반 변환이고 세미 조인으로 풀린 뒤 build 배치도 비용으로 갈려 서브쿼리 쪽이 build가 되는 **HASH JOIN RIGHT SEMI**로도 수행될 수 있으므로 ④가 옳으며, '서브쿼리=후행 고정'·'**IN**=유일화+일반 조인'·'반복 제거니 무조건 변환'은 키워드에서 곧장 건너뛴 반사적 연상입니다.

## 문제 12. 정답 ③

조건절 이행이 만든 새 조건과, 그 조건이 뷰 안으로 들어가는 경로(병합/Pushing)는 별개의 단계입니다.

조건절 이행 :  $A.key = B.key \wedge B.key = 10 \rightarrow A.key = 10$  (바깥 블록에 새 조건 생성)

생성된 새 조건도 다른 조건과 똑같이 ↓ 두 경로로 뷰 안에 도달할 수 있음

경로 1 (병합) : 뷰가 병합되면 한 블록이 되어 자연히 함께 적용

경로 2 (Pushing): 뷰가 병합되지 못해도 조건절 Pushing으로 뷰 안에 밀어 넣음

③은 "이행으로 유도된 조건은 병합될 때에만 뷰로 전달된다"고 못박았습니다. 그러나 이행으로 새로 만들어진 조건도 여느 조건과 다를 바 없는 하나의 술어여서, 뷰가 병합되지 못하더라도 조건절 Pushing의 대상이 되어 뷰 안으로 밀려 들어갈 수 있습니다. '이행은 조건을 만드는 변환이니 병합과 한 몸'이라는 반사적 연상으로, 병합되지 못한 뷰에는 이행 조건이 못 들어간다고 좁혀 버린 진술이라 ③이 부적절합니다.

①②④는 모두 옳습니다.

①: 병합되면 뷰 경계가 사라져 바깥 조건이 뷰 안 테이블의 액세스 조건으로 바로 쓰입니다 — Pushing이라는 별도 단계가 필요 없습니다.

②: 병합이 막힌 뷰에도 Pushing으로 조건을 밀어 넣어 처리 행을 줄이며, 그룹핑 키에 걸린 조건은 집계 전에 걸려져 집계 대상을 줄입니다.

④: 병합과 Pushing은 목적이 다른 변환이고, 병합이 막힌 뷰에 Pushing을 차선택으로 적용하기도 합니다.

| 선택지 | 판정 | 이유                                                                                                          |
|-----|----|-------------------------------------------------------------------------------------------------------------|
| ①   | ○  | 병합되면 뷰 경계가 사라져 바깥 조건이 뷰 안 테이블의 액세스 조건으로 바로 쓰이므로, 별도의 Pushing 단계가 필요 없습니다                                    |
| ②   | ○  | 병합이 막힌 뷰에도 Pushing으로 조건을 밀어 넣어 처리 행을 줄이며, 그룹핑 키에 걸린 조건은 집계 전에 걸려져 집계 대상을 줄입니다                               |
| ③   | ×  | 이행으로 만든 조건도 하나의 술어라 병합되지 못한 뷰에는 조건절 Pushing으로 밀려 들어갈 수 있습니다. '이행=조건 생성=병합과 한 몸'이라는 반사로 Pushing 경로를 지운 진술입니다 |
| ④   | ○  | 병합과 Pushing은 목적이 다른 변환이며, 옵티마이저는 병합이 막힌 뷰에 Pushing을 차선택으로 적용하기도 합니다                                         |

#### ■ 이 문제의 핵심

1. 병합되면 Pushing이 필요 없다. 뷰 경계가 사라져 바깥 조건이 뷰 안 테이블에 바로 걸립니다.
2. 병합이 막혀도 Pushing이 대안. 상수 조건을 뷰 안에 밀어 넣어 처리 행을 줄이고, **GROUP BY** 뷰에서는 집계 전에 걸려 집계 대상을 줄입니다.
3. 이행 조건도 Pushing으로 뷰에 들어간다. 이행이 만든 새 조건은 여느 술어와 같아, 병합되지 못한 뷰에도 밀려 들어갈 수 있습니다.
4. 병합·Pushing·이행은 별개 단계. '이행은 조건 생성이니 병합과 한 몸'은 키워드에 이끌린 반사적 연상입니다.

■ 한 줄 정리 조건절 이행이 만든 조건도 하나의 술어라 병합되지 못한 뷰에는 조건절 Pushing으로 밀려 들어갈 수 있는데, "이행 조건은 병합될 때에만 전달된다"는 ③은 '이행=조건 생성=병합과 한 몸'이라는 반사로 Pushing 경로를 지운 진술이라 부적절합니다.

### 문제 13. 정답 ③

Bind Peeking은 커서를 하드파싱하는 그 순간에 한 번 바인드 값을 엿보아 계획을 세우는 최적화입니다. 오라클은 발행된 SQL 텍스트로 부모 커서를 찾고, 처음 하드파싱할 때 **:b1**에 담긴 실제 값을 들여다본 뒤 그 값의 카디널리티(히스토그램 반영)로 실행계획을 만들어 자식 커서에 넣습니다. 이후 같은 커서를 재사용하는 실행들은 **:b1** 값이 무엇이든 처음 엿본 값 기준으로 만들어진 그 계획을 그대로 씁니다.

여기서 함정이 생깁니다. 편중이 심한 **신고유형**에서 처음 엿본 값이 '일반증여'(97%)였다면 계획은 Full Scan 쪽으로 굳고, 그 커서를 물려받은 '가업승계'(극소수) 실행도 같은 Full Scan을 쓰게 되어 인덱스가 유리한 값에서 오히려 나빠집니다 — 이것이 Bind Peeking의 잘 알려진 부작용입니다.

③은 "바인드를 쓰면 오라클이 히스토그램을 보고 실행마다 그 값에 맞는 계획으로 정규화해 꽃아 매 실행이 각자 최적으로 돈다"고 했는데, 이는 정규화 가정의 오류입니다. 계획은 매 실행이 아니라 첫 하드파싱 때 엿본 한 값 기준으로 만들어져 공유되며, 값별로 계획이 자동으로 갈리지는 않습니다. 값의 선택도에 따라 별도 계획을 나눠 다는 일은 ④의 ACS가 개입해 커서를 bind-aware로 만들었을 때 비로소 일어납니다. 그래서 부적절한 진술은 ③입니다.

①②④는 각각 바인드 방식의 텍스트 동일 → 소프트파싱 재사용(①), 첫 하드파싱 시점의 Bind Peeking(②), ACS의 bind-aware 전환과 값별 자식 커서 분화(④)를 옳게 서술합니다.

| 선택지 | 판정 | 이유                                                                                                                          |
|-----|----|-----------------------------------------------------------------------------------------------------------------------------|
| ①   | ○  | 텍스트가 하나라 부모 커서를 공유하고, 두 번째 발행부터는 자식 커서를 찾아 재사용하는 소프트파싱으로 처리됩니다                                                              |
| ②   | ○  | 히스토그램이 있을 때 첫 하드파싱 순간 <b>:b1</b> 값을 엿보아 그 값의 카디널리티로 계획을 세워 자식 커서에 저장합니다                                                     |
| ③   | ×  | Bind Peeking은 첫 하드파싱 때 한 번만 엿보며, 계획은 그 한 값 기준으로 만들어져 공유됩니다. 히스토그램으로 매 실행을 각자 최적 계획으로 자동 정규화하지 않으므로 "매 실행이 각자 최적"은 성립하지 않습니다 |
| ④   | ○  | ACS는 처음 계획이 다른 선택도의 값에 비효율적일 때 커서를 bind-aware로 표시하고, 값의 선택도별로 별도 자식 커서를 만들어 나눠 실행할 수 있습니다                                   |

#### ■ 이 문제의 핵심

1. 바인드 변수는 텍스트를 하나로 고정합니다. 값이 달라도 부모 커서를 공유하며 두 번째부터 소프트파싱으로 재사용됩니다.

2. Bind Peeking은 첫 하드파싱 때 한 번만 값을 엿봅니다. 그 한 값 기준으로 만든 계획이 이후 실행에 공유되며, 값마다 계획이 자동으로 갈리지 않습니다.
3. 편중 컬럼 + Bind Peeking은 부작용을 낳습니다. 처음 엿본 값에 맞춰 굳은 계획이 다른 선택도의 값에서 비효율적일 수 있습니다.
4. 값별 최적 계획 분화는 ACS의 몫입니다. 커서를 bind-aware로 전환해야 선택도에 따라 별도 자식 커서(다른 계획)가 만들어집니다 — 히스토그램만으로 매 실행이 정규화되지 않습니다.
  - 한 줄 정리 Bind Peeking은 첫 하드파싱 때 한 번 엿본 값으로 계획을 세워 공유하므로, "히스토그램을 보고 매 실행을 각자 최적 계획으로 정규화한다"는 ③은 정규화 가정의 오류라 부적절합니다.

#### 문제 14. 정답 ④

CBO의 결합 카디널리티 추정에는 서로 독립인 여러 축이 얹혀 있습니다. 히스토그램·통계 최신성·버킷 해상도가 각각 무엇을 개선하는지 갈라 봐야 합니다.

- [단일 컬럼 분포] 히스토그램 → 그 '한 컬럼'의 편중 값 선택도를 정밀화  
 [여러 컬럼 결합] CBO 기본 가정 = 컬럼 독립 → 개별 선택도를 그냥 곱함
- 컬럼이 실제로 상관(시·군 ↔ 우편번호)이면 곱셈 결과가 실제와 크게 어긋남
  - 이 상관은 컬럼별 히스토그램·버킷 해상도·통계 최신성 어느 것으로도 메워지지 않음
  - 메우려면 다중 컬럼(확장) 통계 같은 '컬럼 묶음' 통계가 따로 필요

카디널리티가 어긋나는 근본 원인은 컬럼 독립 가정입니다. 컬럼별 히스토그램은 각 컬럼의 분포는 잘 잡아도, 두 컬럼이 함께 움직이는 정도(상관)는 담지 못합니다. 그래서 상관이 강한 컬럼들에 조건을 함께 걸면 선택도의 곱이 실제보다 훨씬 작게(또는 크게) 나옵니다. ④가 이 구조적 한계를 정확히 짚습니다.

나머지는 서로 독립인 두 축을 인과로 엮은 별개 축 혼동입니다.

- ①: 단일 컬럼 분포 정밀도와 결합 추정 정밀도는 다른 축입니다. 히스토그램이 편중 값의 도수를 반영한다는 앞부분은 맞지만, 한 컬럼의 선택도가 아무리 정확해도 결합은 독립 가정 아래 곱해지므로 컬럼 간 상관이 있으면 결합 카디널리티는 여전히 빗나갑니다.
- ②: 통계 최신성과 독립 가정에서 오는 편향은 다른 축입니다. 낡은 통계가 행 수·distinct 값 수를 어긋나게 한다는 앞부분은 맞지만, 통계를 매일 새로 수집해도 곱셈 방식(독립 가정) 자체는 그대로라 상관에서 비롯된 편향은 줄지 않습니다.
- ③: 버킷 해상도(단일 컬럼 분포의 정밀도)와 컬럼 간 상관은 다른 축입니다. 버킷을 촘촘히 잡으면 값 구간이 좁아진다는 앞부분은 맞지만, 그것은 한 컬럼 안 분포를 잘게 나누는 것일 뿐 두 컬럼이 함께 움직이는 관계를 담지 못합니다.

| 선택지 | 판정 | 이유                                                                                                                                                  |
|-----|----|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | 히스토그램이 편중 값의 도수를 반영해 단일 조건 카디널리티를 정밀하게 만든다는 앞부분은 맞지만, 단일 컬럼 정밀도와 결합 정밀도는 독립 축입니다. 재료가 아무리 정확해도 결합은 독립 가정으로 곱해지므로, 시·군과 우편번호처럼 상관이 강하면 결합은 그대로 빗나갑니다 |
| ②   | ×  | 낡은 통계가 행 수·distinct 값 수·최소최대값을 어긋나게 해 개별 선택도를 망친다는 앞부분은 맞지만, 통계 최신성과 독립 가정 편향은 독립 축입니다. 통계를 매일 새로 수집해도 개별 선택도를 곱하는 방식 자체는 그대로라, 상관에서 오는 편향은 줄지 않습니다 |
| ③   | ×  | 버킷을 촘촘히 잡으면 한 버킷의 값 구간이 좁아져 분포가 세밀해진다는 앞부분은 맞지만, 버킷 해상도(단일 컬럼 분포)와 컬럼 간 상관은 독립 축입니다. 그 세밀함은 한 컬럼 안 분포를 잘게 나눌 뿐이라 두 컬럼이 함께 움직이는 관계는 담기지 않습니다         |
| ④   | ○  | CBO는 컬럼 독립을 가정해 개별 선택도를 곱하므로, 상관이 강한 컬럼들에 조건을 함께 걸면 결합 카디널리티가 실제와 어긋나며 컬럼별 히스토그램만으로는 이 상관을 담지 못합니다                                                  |

#### ▪ 이 문제의 핵심

1. CBO는 컬럼 독립을 가정해 선택도를 곱한다. 컬럼 간 상관이 강하면 결합 카디널리티가 실제와 크게 어긋납니다.
2. 컬럼별 히스토그램은 단일 컬럼 분포만 담는다. 두 컬럼이 함께 움직이는 상관은 담지 못해 결합 추정을 바로잡지 못합니다.
3. 통계 최신성 ≠ 독립 가정 편향. 통계를 자주 수집해도 곱셈 방식은 그대로라 상관 편향은 남습니다.
4. 버킷 해상도 ≠ 컬럼 간 상관. 버킷을 촘촘히 잡는 것은 한 컬럼 안 분포를 잘게 나누는 일일 뿐, 컬럼 묶음 통계가 있어야 상관을 반영합니다.
  - 한 줄 정리 CBO는 컬럼 독립을 가정해 개별 선택도를 곱하므로 상관이 강한 컬럼들에 조건을 함께 걸면 결합 카디널리티가 어긋나고 컬럼별 히스토그램만으로는 메울 수 없다는 ④가 옳으며, 단일 컬럼 정밀도·통계 최신성·버킷 해상도를 결합 추정 정밀도로 잇는 진술은 모두 독립인 두 축을 인과로 엮은 혼동입니다.

## 문제 15. 정답 ②

**APPEND** 힌트가 유발하는 직접경로(Direct Path) 적재는 MERGE 안에서 INSERT 부분에만 걸립니다. 미매칭 소스 행은 버퍼 캐시를 거치지 않고 세그먼트의 HWM(고수위선) 위에 새 블록으로 순차 적재되지만, ON 조건으로 매칭된 행에 대한 UPDATE·DELETE는 일반(conventional) 방식으로, 즉 버퍼 캐시를 경유해 기존 블록을 갱신·삭제합니다. 직접경로는 "빈 공간을 재활용하지 않고 위에 쌓는" 적재 최적화이므로 기존 행을 고치는 갱신·삭제에는 적용되지 않습니다. ②는 이 경계를 정확히 서술했으므로 적절합니다.

나머지는 옵션의 동작 경계를 뭉쳤습니다.

①은 병렬 DML의 활성화 경계를 놓쳤습니다. **PARALLEL** 힌트만으로는 질의(스캔) 부분만 병렬화되고, 대상을 바꾸는 DML(변경) 부분은 직렬로 남습니다. 갱신·삭제·삽입이 병렬로 수행되려면 세션에서 **ALTER SESSION ENABLE PARALLEL DML**을 켜거나 **ENABLE\_PARALLEL\_DML** 문장 힌트가 있어야 하는데, 문제의 단서는 이를 켜지 않았다고 못 박았습니다.

③은 LOGGING의 경계를 무시했습니다. 직접경로가 리두를 줄이는 것은 세그먼트가 NOLOGGING이고 DB가 FORCE LOGGING이 아닐 때뿐입니다. 대상이 LOGGING이면 직접경로여도 리두는 정상 생성됩니다.

④는 직접경로의 잠금·가시성 경계를 뒤집었습니다. 직접경로 적재는 대상 테이블에 배타 잠금을 걸어 다른 세션의 일반 DML을 막고, 적재한 트랜잭션도 커밋 전에는 그 테이블을 조회할 수 없습니다(ORA-12838).

| 선택지 | 판정 | 이유                                                                                                  |
|-----|----|-----------------------------------------------------------------------------------------------------|
| ①   | ×  | <b>PARALLEL</b> 힌트만으로는 스캔만 병렬화되고 DML 변경 부분은 직렬입니다. 세션에서 병렬 DML을 켜지 않았으므로 갱신·삭제·삽입은 병렬화되지 않습니다       |
| ②   | ○  | <b>APPEND</b> 직접경로는 INSERT 부분에만 걸려 HWM 위에 순차 적재하고, 매칭 행의 UPDATE·DELETE는 일반 방식(버퍼 캐시 경유)으로 처리됩니다     |
| ③   | ×  | 직접경로가 리두를 줄이려면 세그먼트가 NOLOGGING이어야 합니다. 대상이 LOGGING이라 이 INSERT는 리두를 정상 생성하며 "리두 부하가 사라진다"는 성립하지 않습니다 |
| ④   | ×  | 직접경로 적재는 대상 테이블에 배타 잠금을 걸어 다른 세션의 일반 DML을 막고, 적재 트랜잭션도 커밋 전엔 그 테이블을 조회할 수 없습니다(ORA-12838)           |

### ■ 이 문제의 핵심

1. **APPEND** 직접경로는 MERGE의 INSERT에만 걸립니다. 매칭 행의 UPDATE·DELETE는 일반 방식으로 처리됩니다.
2. **PARALLEL** 힌트만으로는 스캔만 병렬화됩니다. DML 변경 부분을 병렬화하려면 세션에서 병렬 DML을 켜거나 **ENABLE\_PARALLEL\_DML** 힌트가 필요합니다.
3. 직접경로의 리두 절감은 NOLOGGING 조건부입니다. 대상이 LOGGING이면 직접경로여도 리두는 정상 생성됩니다.
4. 직접경로는 테이블 배타 잠금 + 커밋 전 조회 불가입니다. 다른 세션의 일반 DML을 막고, 같은 트랜잭션도 커밋 전엔 방금 넣은 행을 조회할 수 없습니다(ORA-12838).

■ 한 줄 정리 **APPEND** 직접경로는 MERGE의 INSERT 부분에만 걸리고 UPDATE·DELETE는 일반 방식이므로 ②가 적절하며, 병렬 DML 활성화·LOGGING·배타 잠금의 경계를 뭉개 ①③④는 부적절합니다.

## 문제 16. 정답 ③

전체(Full) Partition-Wise Join은 양쪽 테이블이 조인 키로 동일하게 파티셔닝돼 있을 때 성립합니다. "동일하게"란 같은 파티션 키(여기서는 **제휴처번호**), 같은 파티션 방식(HASH), 같은 파티션 수(8개)를 뜻합니다. 이 조건이 갖춰지면 같은 해시 함수가 양쪽의 같은 **제휴처번호**를 같은 파티션 번호로 보내므로, 조인은 대응하는 파티션쌍(a의 P1↔b의 P1, ...)끼리 독립적으로 수행됩니다. 파티션쌍 밖으로 행이 섞일 일이 없어 브로드캐스트·해시 재분배 같은 조인용 데이터 재분배(PX SEND)를 최소화합니다.

[ 파티션-와이즈 조인 성립 판정 ]

- |                                     |                              |   |
|-------------------------------------|------------------------------|---|
| ③ 제휴처번호(조인키=파티션키) · 같은 HASH · 같은 8개 | → 대응 파티션쌍끼리 독립 조인 → 재분배 최소   | ○ |
| ① 이체일자로 조인 (조인키 ≠ 파티션키)             | → 파티션 경계와 조인 키가 무관 → PWJ 불성립 |   |
| ② 한쪽만 파티셔닝                          | → 부분 PWJ: 안 맞는 쪽을 동적 재분배해 맞춤 |   |
| ④ 조인키에 NVL(...) 가공                  | → 키가 가공되면 파티션쌍 대응이 깨짐 → 재분배  |   |

③은 성립 조건을 정확히 짚었으므로 적절합니다.

나머지는 어긋납니다. ①은 조인을 파티션 키(**제휴처번호**)가 아닌 **이체일자로** 걸었습니다 — PWJ는 조인 키와 파티션 키가 같아야 성립하므로, **이체일자** 조인에서는 성립하지 않습니다. ②는 한쪽만 파티셔닝된 경우로, 이때는 파티셔닝이 맞지 않는 쪽을 옵티마이저가 동적으로 재분배(Partial PWJ)해 맞추므로 "재분배가 일어나지 않는다"는 성립하지 않습니다. ④는 값이 같아 보인다는 부분진실로 최선을 최악으로 뒤집습니다. **NVL(제휴처번호, 0)**처럼 파티션 키를 가공하면 옵티마이저가

파티션 경계와 대조할 수 없어 파티션쌍 대응이 깨지고, 결국 재분배로 후퇴합니다.

| 선택지 | 판정 | 이유                                                                                           |
|-----|----|----------------------------------------------------------------------------------------------|
| ①   | ×  | PWJ는 조인 키와 파티션 키가 같아야 성립합니다. <b>이체일자로</b> 조인하면 파티션 키( <b>제휴처번호</b> )와 무관해 파티션쌍 조인이 성립하지 않습니다 |
| ②   | ×  | 한쪽만 파티셔닝되면 부분 PWJ로, 맞지 않는 쪽을 동적으로 재분배해 맞추다. "재분배가 일어나지 않는다"는 성립하지 않습니다                       |
| ③   | ○  | 조인 키 <b>제휴처번호</b> 로 같은 해시·같은 8개 파티션이라 대응 파티션쌍끼리 독립 조인하는 전체 PWJ가 성립해 재분배를 최소화합니다              |
| ④   | ×  | <b>NVL(제휴처번호,0)</b> 으로 파티션 키를 가공하면 파티션쌍 대응이 깨져 PWJ가 무너지고 재분배로 후퇴합니다. "값이 같아 그대로 유지"는 부분진실입니다 |

■ 이 문제의 핵심

1. 전체 PWJ의 성립 조건은 조인 키로 동일 파티셔닝입니다. 같은 파티션 키·방식·개수여야 대응 파티션쌍끼리 독립 조인해 재분배를 최소화합니다.
2. 조인 키 ≠ 파티션 키면 PWJ가 안 됩니다. **이체일자로** 조인하면 파티션 경계와 무관해 성립하지 않습니다.
3. 한쪽만 파티셔닝되면 부분 PWJ입니다. 맞지 않는 쪽을 동적으로 재분배해 맞추므로 재분배가 사라지지는 않습니다.
4. 파티션 키를 가공하면 PWJ가 깨집니다. **NVL**·형변환으로 값이 같아 보여도 파티션쌍 대응이 어긋나 재분배로 후퇴합니다 — "값 보존"은 부분진실입니다.

■ 한 줄 정리 조인 키 **제휴처번호**로 같은 해시·같은 8개 파티션이라 전체 PWJ가 성립하는 ③만 적절하며, 비키 조인(①)·한쪽만 파티셔닝(②)·키 가공(④)은 PWJ가 무너지거나 재분배가 남아 부적절합니다.

### 문제 17. 정답 ③

분배 방식은 각 조인 입력이 어떤 **PX SEND**를 거치는지로 읽습니다. 두 조인의 입력을 각각 봅니다.

업체코드 조인(Id 7) — 두 입력의 SEND를 봅니다.

Id 9 PX SEND BROADCAST :TQ10000 (Q1,00 → 전 서버 복제) 공급업체를 모든 서버에 뿌림  
 Id 11 TABLE ACCESS FULL 공급업체 (약 5,200건) 복제 대상 - 소형  
 Id 12 PX BLOCK ITERATOR (Q1,02 → SEND 없음) 납품이행실적을 제자리에서 읽음  
 Id 13 TABLE ACCESS FULL 납품이행실적 (약 2.6억 건) 재분배 안 함 - 대형

소형 **공급업체**(Id 11) 위에만 **PX SEND BROADCAST**(Id 9)가 있고, 대형 **납품이행실적**(Id 13)은 SEND 없이 **PX BLOCK ITERATOR**(Id 12)로 제자리 스캔합니다. 한쪽에만 BROADCAST가 붙고 다른 쪽은 SEND가 없는 이 조합이 broadcast 분배입니다.

품목코드 조인(Id 4) — 두 입력의 SEND를 봅니다.

Id 6 PX SEND HASH :TQ10002 (Q1,02 → Q1,03) 업체코드 조인 결과를 품목코드 해시로 재분배  
 Id 15 PX SEND HASH :TQ10001 (Q1,01 → Q1,03) 조달품목을 품목코드 해시로 재분배

두 입력 위에 각각 **PX SEND HASH**가 있어, 양쪽을 조인 키(품목코드) 해시로 재분배한 뒤 각 서버가 자기 버킷만 조인합니다. 이것이 hash-hash 분배입니다.

업체코드 조인(Id 7) : 소형 공급업체에만 SEND BROADCAST(Id 9), 대형은 SEND 없음 → broadcast  
 품목코드 조인(Id 4) : 양쪽(Id 6·Id 15)에 각각 SEND HASH → hash-hash

즉 두 조인의 분배가 서로 다릅니다(broadcast + hash-hash). 이를 정확히 지목한 것이 ③입니다. 두 분배를 맞바꾼 ①, broadcast 방향(대형·소형)을 뒤집은 ②, hash-hash를 broadcast로 본 ④는 노드와 어긋납니다.

| 선택지 | 판정 | 이유                                                                                                                                                                                                                                                                             |
|-----|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | :TQ10000·Q1,02라는 흐름 이름과 조달품목이 약 90만 건이라는 앞부분은 맞지만, 두 조인의 분배를 맞바꾼 진술입니다. 업체코드 조인(Id 7)은 :TQ10000에 <b>PX SEND BROADCAST</b> (Id 9)가 붙고 납품이행실적 쪽에는 SEND가 없는 broadcast이고, 품목코드 조인(Id 4)은 Id 6·Id 15 양쪽에 SEND HASH가 붙은 hash-hash입니다 — 두 분배를 서로 바꿔 놓았습니다                             |
| ②   | ×  | Id 9가 :TQ10000을 타는 <b>PX SEND BROADCAST</b> 이고 공급업체가 약 5,200건이며 Id 10이 <b>PX BLOCK ITERATOR</b> 라는 앞부분은 맞지만, Id 9·Id 10은 모두 공급업체(Id 11) 쪽 계보에 얹혀 있어 복제되는 것은 소형 공급업체입니다. 대형 납품이행실적(Id 13)이야말로 Id 12 <b>PX BLOCK ITERATOR</b> 로 SEND 없이 제자리 스캔합니다 — 복제되는 테이블을 대형·소형으로 뒤바꾼 값스왑입니다 |
| ③   | ○  | 업체코드 조인은 소형 공급업체(Id 11)를 BROADCAST(Id 9)로 복제하고 대형 납품이행실적(Id 13)을 로컬로 읽는 broadcast, 품목코드 조인은 Id 6·Id 15 양쪽을 SEND HASH로 재분배하는 hash-hash로, 두 조인의 분배를 정확히 지목했습니다                                                                                                                   |
| ④   | ×  | 업체코드 조인이 소형 공급업체(Id 11)를 Id 9 BROADCAST로 복제하는 broadcast라는 앞부분은 맞지만, 품목코드 조인의 조달품목(Id 17)을 실어 나르는 Id 15는 <b>PX SEND BROADCAST</b> 가 아니라 <b>PX SEND HASH</b> (:TQ10001)이고 업체코드 조인 결과를 올리는 Id 6도 <b>PX SEND HASH</b> (:TQ10002)입니다. 양쪽이 해시로 재분배되는 hash-hash를 broadcast로 본 진술입니다   |

#### ▪ 이 문제의 핵심

1. 분배 방식은 조인 입력마다 **PX SEND**의 유무·종류로 판별합니다. 업체코드 조인은 소형 쪽(Id 9)에만 BROADCAST, 대형 쪽(Id 12)은 SEND 없음 → broadcast입니다.
2. 품목코드 조인은 양쪽(Id 6·Id 15)에 SEND HASH — 조인 키 해시로 재분배해 각 서버가 자기 버킷만 조인하는 hash-hash입니다.
3. 한 플랜 안에서 조인마다 분배가 다를 수 있습니다. 대형×소형(공급업체)은 broadcast, 대형 결과×중형(조달품목)은 hash-hash로 갈렸습니다.
4. 복제 방향과 분배 종류를 뒤바꾸면 오답입니다. BROADCAST는 소형(Id 11) 쪽에 붙고, hash-hash는 양쪽에 SEND HASH가 붙습니다 — 이를 대형 복제(②)·분배 맞교환(①)·전부 broadcast(④)로 보면 노드와 어긋납니다.

▪ 한 줄 정리 업체코드 조인은 소형 공급업체(Id 11)를 **PX SEND BROADCAST**(Id 9)로 복제하고 납품이행실적(Id 13)은 로컬로 읽는 broadcast, 품목코드 조인은 결과(Id 6)와 조달품목(Id 15)을 양쪽 **PX SEND HASH**로 재분배하는 hash-hash로, 한 플랜에 두 분배가 함께 쓰였다.

### 문제 18. 정답 ①

분석함수 안의 **ORDER BY**와, 최종 결과 집합의 출력 순서는 서로 다른 층위입니다.

OVER(ORDER BY ...) : 분석함수를 '계산할 때' 쓰는 내부 정렬 기준 (순위·누적을 매기는 순서)

쿼리 최종 출력 순서 : 오직 바깥(top-level) ORDER BY 로만 보장됨

- 바깥 ORDER BY 가 없으면 결과 행의 순서는 정해지지 않음
- 분석함수의 내부 정렬이 최종 출력 순서를 대신 보장하지 않음

①은 순위 함수 **OVER(ORDER BY ...)**의 내부 정렬이라는 하위 성질을, 최종 출력 순서 보장이라는 상위 결론으로 끌어올렸습니다. **OVER**의 **ORDER BY**는 그 함수의 순위·누적을 계산하는 순서일 뿐이고, 쿼리가 돌려주는 행의 최종 순서는 바깥 **ORDER BY**가 있어야 보장됩니다. 내부 정렬이 곧 결과 정렬이라고 넓힌 하위항목 전체화라 ①이 부적절합니다.

②③④는 모두 옳습니다.

②: 집계형 분석함수는 **ORDER BY**가 없으면 파티션 전체가 한 프레임(파티션 총합), **ORDER BY**가 있으면 기본 프레임이 현재 행까지로 좁혀져 누적합이 됩니다.

③: 분석함수는 **WHERE·GROUP BY·HAVING** 뒤에 계산되므로 **WHERE**에서 바로 조건으로 걸 수 없어, 인라인 뷰로 감싼 뒤 바깥에서 걸려야 합니다.

④: **UNION**은 중복 제거로 Sort Unique/Hash Unique가 따르고 **UNION ALL**은 그 단계 없이 이어 붙이며, 두 방식 모두 브랜치 컬럼 개수·타입 호환을 요구합니다.

| 선택지 | 판정 | 이유                                                                                                                     |
|-----|----|------------------------------------------------------------------------------------------------------------------------|
| ①   | ×  | <b>OVER(ORDER BY ...)</b> 는 분석함수를 계산하는 내부 정렬일 뿐, 최종 출력 순서는 바깥 <b>ORDER BY</b> 로만 보장됩니다. 내부 정렬을 결과 정렬로 넣힌 하위항목 전체화입니다   |
| ②   | ○  | 집계형 분석함수는 <b>ORDER BY</b> 가 없으면 파티션 총합, 있으면 기본 프레임이 현재 행까지로 좁혀져 누적합이 됩니다                                               |
| ③   | ○  | 분석함수는 <b>WHERE · GROUP BY · HAVING</b> 뒤에 계산되어 <b>WHERE</b> 에서 바로 걸 수 없으므로, 인라인 뷰로 감싼 뒤 바깥에서 걸려야 합니다                   |
| ④   | ○  | <b>UNION</b> 은 중복 제거로 Sort Unique/Hash Unique가 따르고 <b>UNION ALL</b> 은 그 단계 없이 이어 붙이며, 두 방식 모두 브랜치 컬럼 개수 · 타입 호환을 요구합니다 |

#### ■ 이 문제의 핵심

1. **OVER**의 **ORDER BY**는 계산용 내부 정렬. 최종 출력 순서는 바깥 **ORDER BY**가 있어야 보장됩니다 — 내부 정렬이 결과 정렬을 대신하지 않습니다.
2. 집계형 분석함수의 기본 프레임. **ORDER BY**가 없으면 파티션 총합, 있으면 현재 행까지의 누적합이 됩니다.
3. 분석함수는 거의 마지막에 계산된다. **WHERE**에서 직접 걸 수 없어 인라인 뷰로 감싸 바깥에서 걸려야 합니다.
4. **UNION**은 Sort Unique, **UNION ALL**은 없음. 두 방식 모두 브랜치 컬럼 개수 · 타입 호환을 요구합니다.

■ 한 줄 정리 **OVER(ORDER BY ...)**는 분석함수를 계산하는 내부 정렬일 뿐 최종 출력 순서는 바깥 **ORDER BY**로만 보장되는데, "내부 정렬 덕분에 바깥 **ORDER BY** 없이도 결과가 정렬되어 나온다"는 ①은 내부 정렬이라는 하위 성질을 결과 정렬 전체로 넣힌 하위항목 전체화라 부적절합니다.

#### 문제 19. 정답 ④

두 격리수준은 스냅샷을 고정하는 시점이 다릅니다. **SERIALIZABLE**은 트랜잭션 시작 시점의 SCN 스냅샷을 트랜잭션 내내 유지하고, **READ COMMITTED**는 각 문장 시작 시점의 최신 커밋 스냅샷으로 읽습니다. 둘 다 커밋된 데이터만 보여 주며(Dirty Read 없음), 미커밋 변경은 Undo로 되감아 커밋 전 값으로 읽습니다.

[ TX2 SELECT(t4) 값 계산 ]

##### ■ SERIALIZABLE - 스냅샷은 t1(트랜잭션 시작) 고정

A : t2에 450으로 COMMIT됐지만 t1보다 나중 → 스냅샷 밖 → Undo로 되감아 400  
 B : t3에 560으로 UPDATE(미커밋) → 커밋 안 됨 → Undo로 되감아 500  
 C : 변경 없음 → 600  
 SUM = 400 + 500 + 600 = 1,500

##### ■ READ COMMITTED - 스냅샷은 t4(문장 시작) 최신 커밋

A : t2에 450으로 COMMIT → 문장 시작 전 커밋분 반영 → 450  
 B : t3에 560으로 UPDATE(미커밋) → 커밋 안 됨 → Undo로 되감아 500  
 C : 변경 없음 → 600  
 SUM = 450 + 500 + 600 = 1,550

**SERIALIZABLE**인 TX2는 스냅샷을 t1에 고정했으므로 t2에 커밋된 A의 변경(450)조차 자기 트랜잭션이 시작된 뒤의 일이라 보지 않고 400으로 읽습니다 → SUM=1,500. 반면 **READ COMMITTED**였다면 t4 문장 시작 시점의 최신 커밋을 반영해 A는 450, 미커밋인 B는 Undo로 되감아 500 → SUM=1,550. 따라서 (**SERIALIZABLE** 1,500 / **READ COMMITTED** 1,550)인 ④가 적절합니다.

①②③은 커밋/미커밋 값과 두 격리수준의 자리를 뒤섞은 값스왑 오답입니다. ①은 두 값의 자리를 통째로 맞바꿨고, ②는 **READ COMMITTED**에서 미커밋인 B(560)까지 반영한 더티 리드(1,610)이며, ③은 **SERIALIZABLE**을 최신 커밋+미커밋 반영으로 오해한 1,610입니다.

| 선택지 | 판정 | 이유                                                                                                    |
|-----|----|-------------------------------------------------------------------------------------------------------|
| ①   | ×  | <b>SERIALIZABLE</b> 1,550 / <b>RC</b> 1,500으로 두 격리수준의 값을 서로 뒤바꿔 놓은 값스왑입니다                             |
| ②   | ×  | <b>SERIALIZABLE</b> 은 1,500이 맞지만, <b>RC</b> 를 1,610(=450+560+600)으로 계산해 미커밋인 B(560)까지 반영한 더티 리드입니다    |
| ③   | ×  | <b>SERIALIZABLE</b> 을 1,610으로 봤는데, 이는 t1 스냅샷을 무시하고 A 커밋(450)+B 미커밋(560)을 반영한 오독입니다                    |
| ④   | ○  | <b>SERIALIZABLE</b> 은 t1 스냅샷 고정이라 1,500(400+500+600), <b>RC</b> 는 t4 최신 커밋 반영이라 1,550(450+500+600)입니다 |

▪ 이 문제의 핵심

1. 스냅샷 고정 시점이 격리수준을 가릅니다. SERIALIZABLE은 트랜잭션 시작 시점, READ COMMITTED는 각 문장 시작 시점입니다.
2. SERIALIZABLE은 시작 뒤의 커밋도 보지 않습니다. t1 이후 t2에 커밋된 A의 변경조차 스냅샷 밖이라 400으로 읽어 SUM=1,500입니다.
3. READ COMMITTED는 문장 시작 전 커밋분을 반영합니다. A는 커밋돼 450, 미커밋 B는 Undo로 되감아 500이라 SUM=1,550입니다.
4. 두 격리수준 모두 미커밋 값은 읽지 않습니다. B의 560은 어느 쪽에서도 보이지 않으며, Undo로 되감아 커밋 전 값 500으로 읽습니다.

▪ 한 줄 정리 SERIALIZABLE은 t1 스냅샷을 고정해 1,500, READ COMMITTED는 t4 최신 커밋을 반영해 1,550이므로 ④가 적절하고, 자리바꿈(①)·더티 리드(②)·스냅샷 무시(③)는 값스왑 오답입니다.

문제 20. 정답 ④

기본 READ COMMITTED에서 RCSI가 꺼져 있으면 조회도 공유(S) 잠금을 요청합니다. 표에서 세션 72는 **B1004**에 S 잠금을 요청하다 세션 60이 켜 X 잠금과 호환되지 않아 **WAIT**에 걸렸습니다. 그런데 세션 60은 아무도 기다리지 않습니다(미커밋 상태로 잠금만 보유). 대기가 72→60 한 방향뿐이라 순환이 없으므로 교착이 아니라 단순 블로킹입니다. 60이 커밋·롤백해 X를 놓으면 72의 S 요청이 곧바로 풀립니다.

[ 표 판독 - 순환 여부 & RCSI 효과 ]

60 : B1004 X GRANT (보유) + 대기 없음

72 : B1004 S WAIT → 60이 켜 X 대기

대기 방향 : 72 → 60 (한 방향, 순환 아님) ⇒ 단순 블로킹

RCSI = OFF : 조회도 S 잠금을 잡음 → X와 충돌 → 72 대기

RCSI = ON : 조회는 잠금 대신 버전 저장소의 '마지막 커밋 버전'을 읽음 → 무대기 진행  
(단, 갱신 세션 60은 RCSI여도 여전히 X 잠금을 잡는다)

RCSI(READ COMMITTED SNAPSHOT)를 켜면 조회는 잠금을 요청하는 대신 tempdb의 버전 저장소에서 60이 커밋하기 직전의 마지막 커밋 버전을 읽습니다. 그러면 X 잠금과 충돌할 일이 없어 72는 대기 없이 진행합니다. ④는 단순 블로킹 판정과 RCSI 전환 효과를 정확히 서술했으므로 적절합니다.

나머지는 어긋납니다. ①은 순환 대기·교착으로 봤지만, 60은 아무것도 기다리지 않아 순환이 없으므로 교착이 아니라 단순 블로킹입니다 — 락 모니터의 victim 롤백은 일어나지 않습니다. ②는 행 버전 관리(RCSI/SNAPSHOT)에서 조회가 무잠금인 것을 갱신까지 무잠금이라고 넓혔습니다 — 버전 관리에서도 UPDATE 같은 쓰기는 여전히 X 잠금을 잡습니다. ③은 "60이 X를 보유"라는, RCSI를 켜든 끄든 공통으로 나타나는 단서(writer는 어느 설정에서나 X를 잡음)를 조회 차단 판별 근거로 삼았습니다. 72의 대기 여부를 가르는 것은 X 보유가 아니라 조회가 S 잠금을 요청하는지(RCSI OFF) 버전을 읽는지(RCSI ON)입니다.

| 선택지 | 판정 | 이유                                                                                              |
|-----|----|-------------------------------------------------------------------------------------------------|
| ①   | ×  | 60은 아무도 기다리지 않아 대기가 72→60 한 방향입니다. 순환이 없으므로 교착이 아니라 단순 블로킹이며 victim 롤백은 없습니다                    |
| ②   | ×  | RCSI/SNAPSHOT에서 무잠금이 되는 것은 조회입니다. 갱신 세션 60의 UPDATE는 버전 관리에서도 X 잠금을 잡으므로 "갱신도 잠금 없이"는 틀립니다       |
| ③   | ×  | "60이 X를 보유"는 RCSI on/off 어디서나 나타나는 공통 단서라 판별 근거가 못 됩니다. 대기는 조회가 S를 요청(OFF)하느냐 버전을 읽(ON)느냐로 갈립니다 |
| ④   | ○  | RCSI OFF라 72의 S 요청이 60의 X에 막힌 단순 블로킹이며, RCSI를 켜면 조회가 버전 저장소의 마지막 커밋 버전을 읽어 블로킹 없이 진행합니다         |

▪ 이 문제의 핵심

1. 한 방향 대기는 단순 블로킹입니다. 60이 아무도 안 기다리므로 순환이 없어 교착이 아니며, 60이 커밋하면 72가 풀립니다.
2. RCSI OFF에서는 조회도 S 잠금을 잡습니다. X와 충돌해 대기가 생기며, 이는 순환 없는 블로킹입니다.
3. RCSI ON이면 조회는 버전을 읽습니다. 잠금 대신 버전 저장소의 마지막 커밋 버전을 읽어 writer의 X와 무관하게 무대기 진행합니다.
4. "writer가 X 보유"는 공통 단서입니다. RCSI on/off 어디서나 나타나므로 조회 차단의 판별 근거가 못 되며, 대기 여부는 조회가 잠금을 요청하는지 버전을 읽는지로 갈립니다.

- 한 줄 정리 대기가 72→60 한 방향이라 순환 없는 단순 블로킹이고 RCSI를 켜면 조희가 버전을 읽어 풀리므로 ④가 적절하며, "writer의 X 보유"라는 공통 단서로 차단을 단정한 ③과 교착 오판(①)·갱신 무잠금 오해(②)는 부적절합니다.